

```
// ===== 文件: lib/main.dart =====
0001 import 'dart:ui';
0002 import 'package:flutter/material.dart';
0003 import 'package:flutter/services.dart';
0004 import 'core/database/storage_service.dart';
0005 import 'core/theme/cyber_colors.dart';
0006 import 'core/utils/cyber_logger.dart';
0007 import 'features/settings/presentation/widgets/privacy_agreement_modal.dart';
0008 import 'features/audio/services/ambient_service.dart';
0009 import 'features/audio/services/sfx_service.dart';
0010 import 'features/dashboard/presentation/dashboard_screen.dart';
0011 import 'features/library/domain/book_model.dart';
0012 import 'features/reader/providers/reader_provider.dart';
0013 import 'features/settings/providers/settings_provider.dart';
0014 import 'package:flutter_riverpod/flutter_riverpod.dart' as riverpod;
0015 import 'shared/widgets/tts_error_listener.dart';
0016 import 'features/audio/services/tts_engine_service.dart';
0018 final GlobalKey<NavigatorState> globalNavigatorKey =
0019     GlobalKey<NavigatorState>();
0021 Future<void> main() async {
0022     WidgetsFlutterBinding.ensureInitialized();
0024     // 全局错误捕获锚点（先注册，确保 Sentry 初始化前的错误也能记录）
0025     FlutterError.onError = CyberLogger.recordFlutterError;
0026     PlatformDispatcher.instance.onError = CyberLogger.recordPlatformError;
0028     // 启动前初始化持久化层与音效引擎
0029     await StorageService.init();
0030     try {
0031         await SfxService.init();
0032     } catch (e, st) {
0033         CyberLogger.recordFlutterError(
0034             FlutterErrorDetails(exception: e, stack: st, library: 'SfxService'),
0035         );
0036     }
0038     // 初始化环境背景音服务
0039     try {
0040         await AmbientService.init();
0041     } catch (e, st) {
0042         CyberLogger.recordFlutterError(
0043             FlutterErrorDetails(exception: e, stack: st, library: 'AmbientService'),
0044         );
0045     }
0047     // 通过 CyberLogger 初始化 Sentry 并启动 App
0048     // DSN 通过 --dart-define=SENTRY_DSN=https://... 注入，空时静默跳过
0049     await CyberLogger.initSentry(
0050         () async => runApp(
0051             const riverpod.ProviderScope(
0052                 child: YueYouApp(),
0053             ),
0054         ),
0055     );
```

```
0056 }
0058 class YueYouApp extends riverpod.ConsumerWidget {
0059   const YueYouApp({super.key});
0061   @override
0062   Widget build(BuildContext context, riverpod.WidgetRef ref) {
0063     return const _Bootstrapper();
0064   }
0065 }
0067 class _Bootstrapper extends riverpod.ConsumerStatefulWidget {
0068   const _Bootstrapper();
0070   @override
0071   riverpod.ConsumerState<_Bootstrapper> createState() => _BootstrapperState();
0072 }
0074 class _BootstrapperState extends riverpod.ConsumerState<_Bootstrapper>
0075   with WidgetsBindingObserver {
0076   bool _booted = false;
0078   /// 记录上一次 SettingsProvider 的 ambient 状态，用于增量对比
0079   bool? _lastAmbientEnabled;
0080   double? _lastAmbientVol;
0081   String? _lastAmbientStyle;
0083   @override
0084   void initState() {
0085     super.initState();
0086     WidgetsBinding.instance
0087       ..addObserver(this)
0088       ..addPostFrameCallback((_) => _checkPrivacyAndBootstrap());
0089   }
0091   @override
0092   void didChangeDependencies() {
0093     super.didChangeDependencies();
0094     // 每当 SettingsProvider 变化时同步 AmbientService
0095     final settings = ref.watch(settingsProvider);
0096     _syncAmbient(settings);
0097   }
0099   /// 同步 SettingsProvider 的 ambient 设置到 AmbientService
0100   void _syncAmbient(SettingsProvider settings) {
0101     if (settings.ambientEnabled != _lastAmbientEnabled) {
0102       _lastAmbientEnabled = settings.ambientEnabled;
0103       AmbientService.setEnabled(settings.ambientEnabled);
0104     }
0105     if (settings.ambientVol != _lastAmbientVol) {
0106       _lastAmbientVol = settings.ambientVol;
0107       AmbientService.setVolume(settings.ambientVol);
0108     }
0109     if (settings.ambientStyle != _lastAmbientStyle) {
0110       _lastAmbientStyle = settings.ambientStyle;
0111       AmbientService.setStyle(settings.ambientStyle);
0112     }
0113   }
0115   /// App 生命周期：切后台暂停，切前台恢复
```

```
0116 @override
0117 void didChangeAppLifecycleState(AppLifecycleState state) {
0118     switch (state) {
0119         case AppLifecycleState.paused:
0120         case AppLifecycleState.inactive:
0121             AmbientService.pause();
0122         case AppLifecycleState.resumed:
0123             AmbientService.resume();
0124         case AppLifecycleState.detached:
0125         case AppLifecycleState.hidden:
0126             AmbientService.dispose();
0127     }
0128 }
0130 Future<void> _checkPrivacyAndBootstrap() async {
0131     if (!mounted) return;
0132     if (!StorageService.hasAgreedPrivacy()) {
0133         // 等待 MaterialApp 内部 Navigator 挂载完成 (最多 5 次, 间隔 50ms)
0134         // 避免首帧时 globalNavigatorKey 尚未绑定导致弹窗被静默跳过
0135         BuildContext? navContext;
0136         for (int i = 0; i < 5; i++) {
0137             navContext = globalNavigatorKey.currentContext;
0138             if (navContext != null) break;
0139             await Future<void>.delayed(const Duration(milliseconds: 50));
0140             if (!mounted) return;
0141         }
0142         if (navContext == null) {
0143             // 极端情况: Navigator 始终未挂载, 记录日志后退出, 绝不进入未授权状态
0144             CyberLogger.captureWarning(
0145                 Exception('隐私弹窗 navContext 重试 5 次仍为 null'),
0146                 tag: 'privacy',
0147                 extra: {'context': '_checkPrivacyAndBootstrap'},
0148             );
0149             SystemNavigator.pop();
0150             return;
0151         }
0152         // navContext 来自 globalNavigatorKey, 由 Navigator 自身管理生命周期,
0153         // 不依赖 _BootstrapperState.mounted; 上方循环已做 mounted 守卫。
0154         // ignore: use_build_context_synchronously
0155         final agreed = await showPrivacyAgreementModal(navContext);
0156         if (!mounted) return;
0157         if (agreed) {
0158             await StorageService.setHasAgreedPrivacy(true);
0159         } else {
0160             return;
0161         }
0162     }
0163     await _bootstrap();
0164 }
0166 Future<void> _bootstrap() async {
0167     if (!_booted) return;
```

```

0168     _booted = true;
0170     final reader = ref.read(readerProvider);
0171     final currentNovelId = StorageService.getCurrentNovelId();
0172     if (currentNovelId == null) return;
0174     final content = await StorageService.loadBookContent(currentNovelId);
0175     if (!mounted || content == null) return;
0177     final List<dynamic> rawLines = content['lines'] as List<dynamic>? ?? [];
0178     final List<dynamic> rawChapters =
0179         content['chapters'] as List<dynamic>? ?? [];
0180     final chapters = rawChapters
0181         .map((e) => ChapterModel.fromJson(e as Map<String, dynamic>))
0182         .toList();
0183     final lines = rawLines.map((e) => e.toString()).toList();
0184     final int initialIndex = StorageService.getCurrentNovelIndex();
0186     await reader.loadPreparedBook(
0187         lines,
0188         bookId: currentNovelId,
0189         chapters: chapters,
0190         initialIndex: initialIndex,
0191     );
0192 }
0194 @override
0195 void dispose() {
0196     WidgetsBinding.instance.removeObserver(this);
0197     super.dispose();
0198 }
0200 @override
0201 Widget build(BuildContext context) {
0202     return MaterialApp(
0203         navigatorKey: globalNavigatorKey,
0204         title: '阅游 YueYou',
0205         debugShowCheckedModeBanner: false,
0206         theme: ThemeData(
0207             scaffoldBackgroundColor: CyberColors.background,
0208             colorScheme: const ColorScheme.dark(
0209                 primary: CyberColors.neonGreen,
0210                 secondary: CyberColors.neonPink,
0211             ),
0212             useMaterial3: true,
0213         ),
0214         builder: (context, child) => Listener(
0215             onPointerDown: (_) => ref.read(ttsEngineProvider).notifyUserActivity(),
0216             behavior: HitTestBehavior.translucent,
0217             child: TtsErrorListener(child: child!),
0218         ),
0219         home: const DashboardScreen(),
0220     );
0221 }
0222 }

```

```
// ===== 文件: lib/core/config/app_info_config.dart =====
0001 /// 应用主体信息配置。
0002 ///
0003 /// 开发者/运营主体和联系方式支持通过 `--dart-define` 覆盖。
0004 /// 默认值为当前个人开发者公开信息。
0005 class AppInfoConfig {
0006   const AppInfoConfig._();
0007   /// 运营主体名称。
0008   static const String developerName = String.fromEnvironment(
0009     'APP_DEVELOPER_NAME',
0010     defaultValue: '胡传龙',
0011   );
0012   /// 隐私与用户反馈联系邮箱。
0013   static const String contactEmail = String.fromEnvironment(
0014     'APP_CONTACT_EMAIL',
0015     defaultValue: 'hucloong@163.com',
0016   );
0017   /// 正式隐私政策页面。
0018   static const String privacyPolicyUrl = String.fromEnvironment(
0019     'PRIVACY_POLICY_URL',
0020     defaultValue: 'https://hclstudio.cn/privacy',
0021   );
0022 }

// ===== 文件: lib/core/config/tts_config.dart =====
0001 /// TTS 配置类
0002 ///
0003 /// 所有服务器地址通过 `--dart-define` 编译时注入，严禁硬编码 IP 或域名。
0004 class TtsConfig {
0005   final String serverUrl;
0006   final Duration requestTimeout;
0007   final int maxRetries;
0008   final Duration baseRetryDelay;
0009   final int maxPrefetchQueue;
0010   const TtsConfig({
0011     required this.serverUrl,
0012     this.requestTimeout = const Duration(seconds: 8),
0013     this.maxRetries = 2,
0014     this.baseRetryDelay = const Duration(milliseconds: 800),
0015     this.maxPrefetchQueue = 6,
0016   });
0017   /// 编译时注入的 TTS 服务器地址 (--dart-define=TTS_SERVER_URL=https://...)
0018   static const String _ttsServerUrl = String.fromEnvironment(
0019     'TTS_SERVER_URL',
0020     defaultValue: 'https://hclstudio.cn/api/v1/tts',
0021   );
0022   /// 编译时注入的书籍 API 基础地址 (--dart-define=BOOK_API_BASE=https://...)
0023   static const String bookApiBase = String.fromEnvironment(
0024     'BOOK_API_BASE',
0025     defaultValue: 'https://hclstudio.cn/api/v1',
```

```

0029 );
0031 // —— 网络层超时常量 ——
0033 /// TTS 本地引擎朗读最长等待时间（防止 FlutterTts 卡死）
0034 static const Duration ttsLocalSpeakTimeout = Duration(seconds: 60);
0036 /// TTS HTTP 下载连接与传输超时
0037 static const Duration ttsDownloadTimeout = Duration(seconds: 15);
0039 /// TTS POST 请求连接超时
0040 static const Duration ttsPostConnectionTimeout = Duration(seconds: 10);
0042 /// TTS POST 响应超时
0043 static const Duration ttsPostResponseTimeout = Duration(seconds: 15);
0045 /// 书籍目录 / 章节 API 请求超时
0046 static const Duration bookApiTimeout = Duration(seconds: 4);
0048 /// 书籍章节 CDN 下载超时
0049 static const Duration bookCdnDownloadTimeout = Duration(seconds: 15);
0051 /// 当前环境配置（编译时确定，零运行时开销）
0052 static const TtsConfig current = TtsConfig(
0053     serverUrl: _ttsServerUrl,
0054 );
0055 }

// ===== 文件：lib/core/constants/book_constants.dart =====
0001 /// 书籍相关常量定义
0002 class BookConstants {
0003     /// 内置默认书籍 ID（西游记），int 类型对齐 BookModel.id
0004     static const int defaultBookId = 999;
0006     /// 内置默认书籍标识字符串（用于分章缓存目录命名）
0007     static const String defaultBookKey = 'xiyouji';
0009     /// 内置默认书籍标题
0010     static const String defaultBookTitle = '西游记';
0012     /// 内置默认书籍作者
0013     static const String defaultBookAuthor = '吴承恩';
0015     /// 内置默认书籍总章节数
0016     static const int defaultTotalChapters = 100;
0018     /// 《西游记》100 回章节标题（离线降级用，索引 0-based 对应第1回至第100回）
0019     static const List<String> xiyoujiChapterTitles = [
0020         '第一回 灵根育孕源流出 心性修持大道生',
0021         '第二回 悟彻菩提真妙理 断魔归本合元神',
0022         '第三回 四海千山皆拱伏 九幽十类尽除名',
0023         '第四回 官封弼马心何足 名注齐天意未宁',
0024         '第五回 乱蟠桃大圣偷丹 反天宫诸神捉怪',
0025         '第六回 观音赴会问原因 小圣施威降大圣',
0026         '第七回 八卦炉中逃大圣 五行山下定心猿',
0027         '第八回 我佛造经传极乐 观音奉旨上长安',
0028         '第九回 袁守诚妙算无私曲 老龙王拙计犯天条',
0029         '第十回 二将军宫门镇鬼 唐太宗地府还魂',
0030         '第十一回 还受生唐王遵善果 度孤魂萧瑀正空门',
0031         '第十二回 玄奘秉诚建大会 观音显象化金蝉',
0032         '第十三回 陷虎穴金星解厄 双叉岭伯钦留僧',
0033         '第十四回 心猿归正 六贼无踪',
0034         '第十五回 蛇盘山诸神暗佑 鹰愁涧意马收缰',

```

0035 '第十六回 观音院僧谋宝贝 黑风山怪窃袈裟',
 0036 '第十七回 孙行者大闹黑风山 观世音收伏熊罴怪',
 0037 '第十八回 观音院唐僧脱难 高老庄大圣除魔',
 0038 '第十九回 云栈洞悟空收八戒 浮屠山玄奘受心经',
 0039 '第二十回 黄风岭唐僧有难 半山中八戒争先',
 0040 '第二十一回 护法设庄留大圣 须弥灵吉定风魔',
 0041 '第二十二回 八戒大战流沙河 木叉奉法收悟净',
 0042 '第二十三回 三藏不忘本 四圣试禅心',
 0043 '第二十四回 万寿山大仙留故友 五庄观行者窃人参',
 0044 '第二十五回 镇元仙赶捉取经僧 孙行者大闹五庄观',
 0045 '第二十六回 孙悟空三岛求方 观世音甘泉活树',
 0046 '第二十七回 尸魔三戏唐三藏 圣僧恨逐美猴王',
 0047 '第二十八回 花果山群妖聚义 黑松林三藏逢魔',
 0048 '第二十九回 脱难江流来国土 承恩八戒转山林',
 0049 '第三十回 邪魔侵正法 意马忆心猿',
 0050 '第三十一回 猪八戒义激猴王 孙行者智降妖怪',
 0051 '第三十二回 平顶山功曹传信 莲花洞木母逢灾',
 0052 '第三十三回 外道迷真性 元神助本心',
 0053 '第三十四回 魔王巧算困心猿 大圣腾挪骗宝贝',
 0054 '第三十五回 外道施威欺正性 心猿获宝伏邪魔',
 0055 '第三十六回 心猿正处诸缘伏 劈破旁门见月明',
 0056 '第三十七回 鬼王夜谒唐三藏 悟空神化引婴儿',
 0057 '第三十八回 婴儿问母知邪正 金木参玄见假真',
 0058 '第三十九回 一粒金丹天上得 三年故主世间生',
 0059 '第四十回 婴儿戏化禅心乱 猿马刀圭木母空',
 0060 '第四十一回 心猿遭火败 木母被魔擒',
 0061 '第四十二回 大圣殷勤拜南海 观音慈善缚红孩',
 0062 '第四十三回 黑河妖孽擒僧去 西洋龙子捉鼉回',
 0063 '第四十四回 法身元运逢车力 心正妖邪度脊关',
 0064 '第四十五回 三清观大圣留名 车迟国猴王显法',
 0065 '第四十六回 外道弄强欺正法 心猿显圣灭诸邪',
 0066 '第四十七回 圣僧夜阻通天水 金木垂慈救小童',
 0067 '第四十八回 魔弄寒风飘大雪 僧思拜佛履层冰',
 0068 '第四十九回 三藏有灾沉水宅 观音救难现鱼篮',
 0069 '第五十回 情因旧恨生灾毒 心主遭魔幸破光',
 0070 '第五十一回 心猿空用千般计 水火无功难炼魔',
 0071 '第五十二回 悟空大闹金兜洞 如来暗示主人公',
 0072 '第五十三回 禅主吞餐怀鬼孕 黄婆运水解邪胎',
 0073 '第五十四回 法性西来逢女国 心猿定计脱烟花',
 0074 '第五十五回 色邪淫戏唐三藏 性正归真胜女色',
 0075 '第五十六回 神狂诛草寇 道昧放心猿',
 0076 '第五十七回 真行者落伽山诉苦 假猴王水帘洞誉文',
 0077 '第五十八回 二心搅乱大乾坤 一体难修真寂灭',
 0078 '第五十九回 唐三藏路阻火焰山 孙行者一调芭蕉扇',
 0079 '第六十回 牛魔王罢战赴华筵 孙行者二调芭蕉扇',
 0080 '第六十一回 猪八戒助力败魔王 孙行者三调芭蕉扇',
 0081 '第六十二回 涤垢洗心惟扫塔 缚魔归正乃修身',
 0082 '第六十三回 二僧荡涤波月洞 胜法降伏无底船',
 0083 '第六十四回 荆棘岭悟能努力 木仙庵三藏谈诗',
 0084 '第六十五回 妖邪假设小雷音 四众皆遭大厄难',

```

0085      '第六十六回 诸神遭毒手 弥勒缚妖魔',
0086      '第六十七回 拯救驼罗庄 消除疥癬病',
0087      '第六十八回 朱紫国唐僧论前世 孙行者施为三折肱',
0088      '第六十九回 心主夜间修药物 君王筵上论妖邪',
0089      '第七十回 妖魔宝放烟沙火 悟空计盗紫金铃',
0090      '第七十一回 行者假名降怪狴 观音现像伏金毛',
0091      '第七十二回 盘丝洞七情迷本 濯垢泉八戒忘形',
0092      '第七十三回 情因旧恨盘丝洞 壁脱香风蜈蚣精',
0093      '第七十四回 长庚传报魔头狠 行者施为变化能',
0094      '第七十五回 心猿钻入魔王腹 将法三番数字成',
0095      '第七十六回 心神居舍魔归性 木母同降怪体真',
0096      '第七十七回 群魔欺本性 一体拜真如',
0097      '第七十八回 比丘怜子遣阴神 金殿识魔谈道德',
0098      '第七十九回 寻洞擒妖逢老寿 当朝正主救婴儿',
0099      '第八十回 姹女育阳求配偶 心猿护主识妖邪',
0100      '第八十一回 镇海寺心猿知怪 黑松林三众寻师',
0101      '第八十二回 姹女求阳 元神护道',
0102      '第八十三回 心猿识得丹头 姹女还归本性',
0103      '第八十四回 难灭伽持圆大觉 法王成正体天然',
0104      '第八十五回 心猿妒木母 魔主计吞禅',
0105      '第八十六回 木母助威征怪物 金公施法灭妖邪',
0106      '第八十七回 凤仙郡冒天止雨 孙大圣劝善施霖',
0107      '第八十八回 禅到玉华施法会 心猿木母授门人',
0108      '第八十九回 黄狮精虎力大 白鹿精太乙真',
0109      '第九十回 师狮授受同归一 盗宝欺人反受殃',
0110      '第九十一回 金平府元夜观灯 玄英洞唐僧供状',
0111      '第九十二回 三僧大战青龙山 四星挟捉犀牛怪',
0112      '第九十三回 给孤园问古怀乡 天竺国朝王遇偶',
0113      '第九十四回 四僧宴乐御花园 一怪空怀情欲喜',
0114      '第九十五回 假合真形擒玉兔 真阴归正会灵元',
0115      '第九十六回 寇员外喜待高僧 唐长老不贪富贵',
0116      '第九十七回 金酬外护遭魔蛰 圣显幽魂救本原',
0117      '第九十八回 猿熟马驯方论正 功完行满见真如',
0118      '第九十九回 九九数完魔灭尽 三三行满道归根',
0119      '第一百回 径回东土 五圣成真',
0120  ];
0121 }

```

```
// ===== 文件: lib/core/constants/cyber_error_messages.dart =====
```

```

0001 class CyberErrorMessages {
0002     // =====
0003     // 网络与云端通讯错误（屏蔽底层实现细节）
0004     // =====
0005     static const String ttsNodeUnresponsive = '语音服务暂不可用，请检查网络后重试';
0006     static const String ttsConnectTimeout = '服务连接超时，请检查网络设置';
0007     static const String ttsRequestTimeout = '请求处理超时，请稍后重试';
0008     static const String ttsAudioLoadTimeout = '语音加载缓慢，为您切换至本地语音';
0010     static String ttsServerErrorCode(int code) => '服务器繁忙，请稍后重试（错误码: $code)';
0012     // =====
0013     // 降级与兜底提醒（用户友好的状态变更）

```



```

0014 // =====
0015 static const String ttsFallbackDisconnected = '云端连接因网络波动断开，已自动切换为本地语音';
0016 static const String ttsFallbackTimeout = '云端资源加载失败，正在使用本地语音播放';
0018 // =====
0019 // 协议与解析异常（隐藏服务架构与堆栈）
0020 // =====
0021 static const String ttsInvalidFormat = '语音数据异常，请确保应用为最新版或稍后重试';
0022 static const String ttsNotJsonObject = '语音服务暂不支持该解析，请稍后重试';
0024 static String ttsMissingUrl(String body) => '获取语音地址失败，请重新尝试';
0025 static String ttsMissingUrlTest(String body) => '语音服务通道发生异常，请联系开发者获取支持';
0027 // =====
0028 // 业务逻辑与系统状态提示
0029 // =====
0030 static const String ttsRequireBookFirst = '请先导入一部书籍，再开启语音朗读';
0031 static const String ttsNoContentRequiresBook = '当前没有内容可以朗读，请选择一部书籍';
0032 static const String ttsIdleTimeout = '您已经休息很久啦，为了省电，语音已自动暂停';
0033 static const String ttsAudioParamFailed = '语音参数调节失败，请重启功能重试';
0035 // =====
0036 // 本地数据与文件导入
0037 // =====
0038 static const String importFormatFailed = '不支持该书籍的格式，请确认是否为文本文件';
0039 static const String importReadPathFailed = '无法读取文件路径，请检查您的存储权限';
0040 static const String testFailedUnresponsive = '连通性测试未通过：网络连接异常，服务不可用';
0042 static String importFileTooLarge(int maxMb) =>
0043     '为了保证流畅阅读，暂不支持体积超过 ${maxMb}MB 的书籍，建议分卷导入';
0044 }

```

```
// ===== 文件: lib/core/database/i_storage_service.dart =====
```

```

0001 /// 本地持久化服务的统一接口定义（依赖倒置原则）
0002 ///
0003 /// 所有业务逻辑层应依赖本接口，而非直接引用 [StorageService] 静态类。
0004 /// 这样可在测试中注入 Mock 实现，在生产环境中注入 [LocalStorageService]，
0005 /// 实现业务逻辑与具体存储实现的完全解耦。
0006 ///
0007 /// ## 设计约束
0008 /// - 本接口位于 `core/database/`，仅允许被 `features/*/providers/` 消费。
0009 /// - 严禁在 `presentation/` 层直接调用任何存储接口。
0010 /// - 所有写操作均为 async (`Future<void>`)，读操作可同步或异步。
0011 abstract interface class IStorageService {
0012     // —— 初始化 ——
0014     /// 初始化底层存储引擎（应在 [runApp] 前调用一次）
0015     Future<void> init();
0017     // —— 游戏状态 ——
0019     /// 保存完整的 2048 游戏快照
0020     ///
0021     /// - [board]: 4×4 棋盘（`null` 表示空格，非 `null` 为 `{id, value}`）
0022     /// - [score]: 当前全盘总分（全盘求和法，非累加法）
0023     /// - [combo]: 当前连击数
0024     /// - [bestScore]: 历史最佳得分
0025     /// - [maxCombo]: 历史最大连击

```

```
0026  /// - [novelIndex]: 当前关联的小说索引
0027  /// - [currentNovelId]: 当前关联的小说 ID (可为 null)
0028  Future<void> saveGameState({
0029      required List<List<Map<String, dynamic?>>> board,
0030      required int score,
0031      required int combo,
0032      required int bestScore,
0033      required int maxCombo,
0034      required int novelIndex,
0035      String? currentNovelId,
0036  });
0037  /// 读取游戏快照, 返回 `null` 表示无存档或存档损坏
0038  Map<String, dynamic?> loadGameState();
0039  /// 读取历史最佳得分 (无存档时返回 0)
0040  int loadBestScore();
0041  /// 读取历史最大连击数 (无存档时返回 0)
0042  int loadMaxCombo();
0043  // —— 书架元数据 ——
0044  /// 保存书架元数据列表 (全量覆盖写入)
0045  ///
0046  /// 每个元素为书籍元数据 Map, 至少包含 `id`、`title`、`total` 字段。
0047  Future<void> saveBookshelf(List<Map<String, dynamic>> shelf);
0048  /// 读取书架元数据列表 (损坏或空时返回空列表)
0049  List<Map<String, dynamic>> loadBookshelf();
0050  // —— 阅读进度 ——
0051  /// 更新指定书籍的阅读进度
0052  ///
0053  /// - [bookId]: 书籍唯一标识
0054  /// - [cursor]: 当前行号 (0-based)
0055  /// - [total]: 总行数 (≤ 0 时忽略, 防止除零)
0056  Future<void> updateReadingRecord(String bookId, int cursor, int total);
0057  /// 读取指定书籍的阅读进度
0058  ///
0059  /// 返回 `[cursor, total, percent]`, 无记录时返回默认零进度。
0060  Map<String, dynamic> getReadingRecord(String bookId);
0061  /// 删除指定书籍的阅读进度记录
0062  Future<void> deleteReadingRecord(String bookId);
0063  // —— 当前小说状态 ——
0064  /// 设置当前激活的小说 ID (传入 `null` 时清除)
0065  Future<void> setCurrentNovelId(String? id);
0066  /// 读取当前激活的小说 ID (未设置时返回 `null`)
0067  String? getCurrentNovelId();
0068  /// 读取当前激活的小说在书架中的索引 (未设置时返回 0)
0069  int getCurrentNovelIndex();
0070  /// 设置当前激活的小说在书架中的索引
0071  Future<void> setCurrentNovelIndex(int index);
0072  // —— 书籍正文内容 ——
0073  /// 将书籍正文按行存储为 JSON 文件 (大文件, 存入 Documents 目录)
0074  ///
0075  /// - [bookId]: 书籍唯一标识 (作为文件名)
```

```
0093  /// - [lines]: 全文按行分割后的字符串列表
0094  /// - [chapters]: 章节目录, 每项至少含 `title`、`lineIndex` 字段
0095  Future<void> saveBookContent(
0096      String bookId, {
0097      required List<String> lines,
0098      required List<Map<String, dynamic>> chapters,
0099  });
0101  /// 读取书籍正文内容
0102  ///
0103  /// 返回 `{lines: List<String>, chapters: List<Map>}` ,
0104  /// 文件不存在或 JSON 损坏时返回 `null`。
0105  Future<Map<String, dynamic>?> loadBookContent(String bookId);
0107  /// 删除指定书籍的正文文件 (文件不存在时静默忽略)
0108  Future<void> deleteBookContent(String bookId);
0110  // —— 全局设置 ——
0112  /// 读取音效开关 (默认 `true`)
0113  bool getSettingSound();
0115  /// 设置音效开关并持久化
0116  Future<void> setSettingSound(bool v);
0118  /// 读取 TTS 朗读开关 (默认 `true`)
0119  bool getSettingStoryTts();
0121  /// 设置 TTS 朗读开关并持久化
0122  Future<void> setSettingStoryTts(bool v);
0124  /// 读取 TTS 音色 ID (默认 `zh-CN-XiaoxiaoNeural`)
0125  String getSettingVoice();
0127  /// 设置 TTS 音色 ID 并持久化
0128  Future<void> setSettingVoice(String v);
0130  /// 读取空闲超时 (分钟, 默认 1)
0131  int getSettingIdleTimeout();
0133  /// 设置空闲超时并持久化
0134  Future<void> setSettingIdleTimeout(int v);
0136  /// 读取 TTS 播放倍速 (默认 `1.0`)
0137  double getSettingTtsRate();
0139  /// 设置 TTS 播放倍速并持久化
0140  Future<void> setSettingTtsRate(double v);
0142  /// 读取环境音量 (默认 `0.5`, 范围 `0.0` - `1.0`)
0143  double getSettingAmbientVol();
0145  /// 设置环境音量并持久化
0146  Future<void> setSettingAmbientVol(double v);
0148  /// 读取环境背景音乐开关 (默认 `true`)
0149  bool getSettingAmbientEnabled();
0151  /// 设置环境背景音乐开关并持久化
0152  Future<void> setSettingAmbientEnabled(bool v);
0154  // —— 隐私协议 ——
0156  /// 读取用户是否已同意隐私协议 (默认 `false`)
0157  bool hasAgreedPrivacy();
0159  /// 设置隐私协议同意状态并持久化
0160  Future<void> setHasAgreedPrivacy(bool v);
0162  // —— 粘性位: 用户是否曾主动选择过书籍 ——
0164  /// 读取用户是否曾主动选择过书籍 (默认 `false`)
```

```

0165  ///
0166  /// 用于区分"从未有过书籍的新用户"与"删掉所有书籍的老用户",
0167  /// 仅前者在书架为空时触发内置默认书籍自动注入。
0168  bool hasSelectedBook();
0169  /// 设置粘性位并持久化
0170  Future<void> setHasSelectedBook(bool v);
0171  // —— 分章文本缓存（文件路径，非 SharedPreferences）
0172  /// 将指定书籍的某章节纯文本写入本地文件
0173  ///
0174  /// 文件路径: `{documentsDir}/books/chapters/{bookId}/{chapterIndex:03d}.txt`
0175  Future<void> saveChapterCache(String bookId, int chapterIndex, String text);
0176  /// 读取指定书籍章节的本地缓存文本
0177  ///
0178  /// 文件不存在时返回 `null`。
0179  Future<String?> loadChapterCache(String bookId, int chapterIndex);
0180  /// 删除指定书籍所有已缓存章节文件
0181  Future<void> clearChapterCache(String bookId);
0182  /// LRU 清理: 仅保留 `currentChapterIndex ± keepAround` 范围内的章节缓存
0183  ///
0184  /// 用于防止长时间阅读后缓存文件堆积。
0185  Future<void> pruneChapterCache(
0186    String bookId,
0187    int currentChapterIndex, {
0188      int keepAround = 3,
0189    });
0190  // —— 书目录缓存（文件路径）
0191  /// 将书目录（章节标题 + 索引列表）序列化写入本地文件
0192  ///
0193  /// 文件路径: `{documentsDir}/books/catalogs/{bookId}_catalog.json`
0194  Future<void> saveBookCatalog(
0195    String bookId,
0196    List<Map<String, dynamic>> chapters,
0197  );
0198  /// 读取书目录缓存
0199  ///
0200  /// 文件不存在或 JSON 损坏时返回 `null`。
0201  Future<List<Map<String, dynamic>>> loadBookCatalog(String bookId);
0202 }

// ===== 文件: lib/core/database/local_storage_service.dart =====
0001 import 'package:yueyou/core/database/i_storage_service.dart';
0002 import 'package:yueyou/core/database/storage_service.dart';
0003 /// [IStorageService] 的本地持久化实现（生产环境注入版本）
0004 ///
0005 /// 内部将所有调用委托给 [StorageService] 静态方法，保持与现有
0006 /// SharedPreferences + path_provider 存储行为 100% 一致。
0007 ///
0008 /// ## 用法
0009 /// ```dart
0010 /// // 在 main.dart 中完成初始化

```

```

0012 /// final storage = LocalStorageService();
0013 /// await storage.init();
0014 /// runApp(MultiProvider(providers: [
0015 ///   Provider<IStorageService>.value(value: storage),
0016 ///   // ... 其他 Provider
0017 /// ]));
0018 /// ```
0019 ///
0020 /// ## 测试替换
0021 /// 在测试中注入 [MockStorageService] (实现 [IStorageService]) ,
0022 /// 无需依赖 SharedPreferences, 实现完全隔离。
0023 class LocalStorageService implements IStorageService {
0024   // —— 初始化 ——
0025   @override
0026   Future<void> init() => StorageService.init();
0027   // —— 游戏状态 ——
0028   @override
0029   Future<void> saveGameState({
0030     required List<List<Map<String, dynamic?>>> board,
0031     required int score,
0032     required int combo,
0033     required int bestScore,
0034     required int maxCombo,
0035     required int novelIndex,
0036     String? currentNovelId,
0037   }) =>
0038     StorageService.saveGameState(
0039       board: board,
0040       score: score,
0041       combo: combo,
0042       bestScore: bestScore,
0043       maxCombo: maxCombo,
0044       novelIndex: novelIndex,
0045       currentNovelId: currentNovelId,
0046     );
0047   @override
0048   Map<String, dynamic>? loadGameState() => StorageService.loadGameState();
0049   @override
0050   int loadBestScore() => StorageService.loadBestScore();
0051   @override
0052   int loadMaxCombo() => StorageService.loadMaxCombo();
0053   // —— 书架元数据 ——
0054   @override
0055   Future<void> saveBookshelf(List<Map<String, dynamic>> shelf) =>
0056     StorageService.saveBookshelf(shelf);
0057   @override
0058   List<Map<String, dynamic>> loadBookshelf() => StorageService.loadBookshelf();
0059   // —— 阅读进度 ——
0060   @override
0061   Future<void> updateReadingRecord(String bookId, int cursor, int total) =>

```

```
0073         StorageService.updateReadingRecord(bookId, cursor, total);
0075     @override
0076     Map<String, dynamic> getReadingRecord(String bookId) =>
0077         StorageService.getReadingRecord(bookId);
0079     @override
0080     Future<void> deleteReadingRecord(String bookId) =>
0081         StorageService.deleteReadingRecord(bookId);
0083     // —— 当前小说状态 ——
0085     @override
0086     Future<void> setCurrentNovelId(String? id) =>
0087         StorageService.setCurrentNovelId(id);
0089     @override
0090     String? getCurrentNovelId() => StorageService.getCurrentNovelId();
0092     @override
0093     int getCurrentNovelIndex() => StorageService.getCurrentNovelIndex();
0095     @override
0096     Future<void> setCurrentNovelIndex(int index) =>
0097         StorageService.setCurrentNovelIndex(index);
0099     // —— 书籍正文内容 ——
0101     @override
0102     Future<void> saveBookContent(
0103         String bookId, {
0104             required List<String> lines,
0105             required List<Map<String, dynamic>> chapters,
0106         }) =>
0107         StorageService.saveBookContent(bookId, lines: lines, chapters: chapters);
0109     @override
0110     Future<Map<String, dynamic>?> loadBookContent(String bookId) =>
0111         StorageService.loadBookContent(bookId);
0113     @override
0114     Future<void> deleteBookContent(String bookId) =>
0115         StorageService.deleteBookContent(bookId);
0117     // —— 全局设置 ——
0119     @override
0120     bool getSettingSound() => StorageService.getSettingSound();
0122     @override
0123     Future<void> setSettingSound(bool v) => StorageService.setSettingSound(v);
0125     @override
0126     bool getSettingStoryTts() => StorageService.getSettingStoryTts();
0128     @override
0129     Future<void> setSettingStoryTts(bool v) =>
0130         StorageService.setSettingStoryTts(v);
0132     @override
0133     String getSettingVoice() => StorageService.getSettingVoice();
0135     @override
0136     Future<void> setSettingVoice(String v) => StorageService.setSettingVoice(v);
0138     @override
0139     int getSettingIdleTimeout() => StorageService.getSettingIdleTimeout();
0141     @override
0142     Future<void> setSettingIdleTimeout(int v) =>
```

```
0143         StorageService.setSettingIdleTimeout(v);
0144     @override
0145     double getSettingTtsRate() => StorageService.getSettingTtsRate();
0146     @override
0147     Future<void> setSettingTtsRate(double v) =>
0148         StorageService.setSettingTtsRate(v);
0149     @override
0150     double getSettingAmbientVol() => StorageService.getSettingAmbientVol();
0151     @override
0152     Future<void> setSettingAmbientVol(double v) =>
0153         StorageService.setSettingAmbientVol(v);
0154     @override
0155     bool getSettingAmbientEnabled() => StorageService.getSettingAmbientEnabled();
0156     @override
0157     Future<void> setSettingAmbientEnabled(bool v) =>
0158         StorageService.setSettingAmbientEnabled(v);
0159     // —— 隐私协议 ——
0160     @override
0161     bool hasAgreedPrivacy() => StorageService.hasAgreedPrivacy();
0162     @override
0163     Future<void> setHasAgreedPrivacy(bool v) =>
0164         StorageService.setHasAgreedPrivacy(v);
0165     // —— 粘性位：用户是否曾主动选择过书籍 ——
0166     @override
0167     bool hasSelectedBook() => StorageService.hasSelectedBook();
0168     @override
0169     Future<void> setHasSelectedBook(bool v) =>
0170         StorageService.setHasSelectedBook(v);
0171     // —— 分章文本缓存 ——
0172     @override
0173     Future<void> saveChapterCache(String bookId, int chapterIndex, String text) =>
0174         StorageService.saveChapterCache(bookId, chapterIndex, text);
0175     @override
0176     Future<String?> loadChapterCache(String bookId, int chapterIndex) =>
0177         StorageService.loadChapterCache(bookId, chapterIndex);
0178     @override
0179     Future<void> clearChapterCache(String bookId) =>
0180         StorageService.clearChapterCache(bookId);
0181     @override
0182     Future<void> pruneChapterCache(
0183         String bookId,
0184         int currentChapterIndex, {
0185         int keepAround = 3,
0186     }) =>
0187         StorageService.pruneChapterCache(
0188             bookId,
0189             currentChapterIndex,
0190             keepAround: keepAround,
0191         );
0192     // —— 书目缓存 ——
```

```

0212 @override
0213 Future<void> saveBookCatalog(
0214     String bookId,
0215     List<Map<String, dynamic>> chapters,
0216 ) =>
0217     StorageService.saveBookCatalog(bookId, chapters);
0218 @override
0219 Future<List<Map<String, dynamic>>?> loadBookCatalog(String bookId) =>
0220     StorageService.loadBookCatalog(bookId);
0221
0222 }

```

// ===== 文件: lib/core/database/storage_service.dart =====

```

0001 import 'dart:convert';
0002 import 'dart:io';
0003 import 'package:flutter/foundation.dart';
0004 import 'package:path_provider/path_provider.dart';
0005 import 'package:shared_preferences/shared_preferences.dart';
0006 import 'package:yueyou/core/utils/cyber_logger.dart';
0007
0008 /// 全量本地持久化服务
0009 /// 完整复刻旧版 localStorage + LocalDB.js 的所有存储行为
0010 /// 键名与旧版 JS 100% 对齐, 保证未来双端迁移零成本
0011 class StorageService {
0012     // —— 键名 (1:1 对应 JS localStorage key) ——
0013     static const String _kBestScore = 'bestScore_premium';
0014     static const String _kMaxCombo = 'maxCombo';
0015     static const String _kLocalSave = 'local_save_data';
0016     static const String _kBookshelf = 'local_bookshelf';
0017     static const String _kReadingRecords = 'reading_records';
0018     static const String _kCurrentNovelId = 'current_novel_id';
0019     static const String _kNovelIndex = 'novel_index';
0020     static const String _kSettingSound = 'setting_sound';
0021     static const String _kSettingStoryTts = 'setting_story_tts';
0022     static const String _kSettingVoice = 'setting_voice';
0023     static const String _kSettingIdleTimeout = 'setting_idle_timeout';
0024     static const String _kSettingTtsRate = 'setting_tts_rate';
0025     static const String _kSettingAmbientVol = 'setting_ambient_vol';
0026     static const String _kSettingAmbientEnabled = 'setting_ambient_enabled';
0027     static const String _kSettingAmbientStyle = 'setting_ambient_style';
0028     static const String _kHasAgreedPrivacy = 'has_agreed_privacy';
0029     static const String _kHasSelectedBook = 'user_has_selected_book';
0030     static const String _kCurrentChapterIndex = 'current_chapter_index';
0031     static const String _kSettingAnimationQuality = 'setting_animation_quality';
0032     static SharedPreferences? _prefs;
0033     static Future<void> init() async {
0034         _prefs ??= await SharedPreferences.getInstance();
0035     }
0036
0037     /// 测试专用: 清除 SharedPreferences 单例缓存, 使 [init] 可重新初始化
0038     /// 生产环境禁止调用
0039     @visibleForTesting
0040     static void resetForTesting() => _prefs = null;

```



```

0044 static SharedPreferences get _p {
0045     assert(_prefs != null, 'StorageService.init() 必须在 runApp 前调用');
0046     return _prefs!;
0047 }
0048 // —— 游戏状态 (local_save_data) ——
0049 /// 对应 JS saveLocalState() 的完整快照
0050 static Future<void> saveGameState({
0051     required List<List<Map<String, dynamic>?>> board,
0052     required int score,
0053     required int combo,
0054     required int bestScore,
0055     required int maxCombo,
0056     required int novelIndex,
0057     String? currentNovelId,
0058 }) async {
0059     final st = {
0060         'board_data': jsonEncode(board),
0061         'score': score,
0062         'combo': combo,
0063         'bestScore': bestScore,
0064         'maxCombo': maxCombo,
0065         'novel_index': novelIndex,
0066         'current_novel_id': currentNovelId,
0067     };
0068     await _p.setString(_kLocalSave, jsonEncode(st));
0069     await _p.setInt(_kBestScore, bestScore);
0070     await _p.setInt(_kMaxCombo, maxCombo);
0071     await _p.setInt(_kNovelIndex, novelIndex);
0072     if (currentNovelId != null) {
0073         await _p.setString(_kCurrentNovelId, currentNovelId);
0074     }
0075 }
0076 static Map<String, dynamic>? loadGameState() {
0077     final saved = _p.getString(_kLocalSave);
0078     if (saved == null) return null;
0079     try {
0080         return jsonDecode(saved) as Map<String, dynamic>;
0081     } catch (_) {
0082         return null;
0083     }
0084 }
0085 static int loadBestScore() => _p.getInt(_kBestScore) ?? 0;
0086 static int loadMaxCombo() => _p.getInt(_kMaxCombo) ?? 0;
0087 // —— 书架元数据 (local_bookshelf) ——
0088 static Future<void> saveBookshelf(List<Map<String, dynamic>> shelf) async {
0089     await _p.setString(_kBookshelf, jsonEncode(shelf));
0090 }
0091 static List<Map<String, dynamic>> loadBookshelf() {
0092     final s = _p.getString(_kBookshelf);
0093     if (s == null) return [];

```

```

0099     try {
0100         return (jsonDecode(s) as List).cast<Map<String, dynamic>>();
0101     } catch (_) {
0102         return [];
0103     }
0104 }
0106 // —— 阅读进度 (reading_records / ProgressManager) ——
0107 static Future<void> updateReadingRecord(
0108     String bookId,
0109     int cursor,
0110     int total,
0111 ) async {
0112     if (total <= 0) return;
0113     final records = _loadReadingRecords();
0114     final percent = (cursor / total * 100).clamp(0.0, 100.0);
0115     records[bookId] = {
0116         'cursor': cursor,
0117         'total': total,
0118         'percent': double.parse(percent.toStringAsFixed(2)),
0119     };
0120     await _p.setString(_kReadingRecords, jsonEncode(records));
0121 }
0123 static Map<String, dynamic> getReadingRecord(String bookId) {
0124     final records = _loadReadingRecords();
0125     return records[bookId] as Map<String, dynamic>? ??
0126         {'cursor': 0, 'total': 1, 'percent': 0.0};
0127 }
0129 static Future<void> deleteReadingRecord(String bookId) async {
0130     final records = _loadReadingRecords();
0131     records.remove(bookId);
0132     await _p.setString(_kReadingRecords, jsonEncode(records));
0133 }
0135 static Map<String, dynamic> _loadReadingRecords() {
0136     final s = _p.getString(_kReadingRecords);
0137     if (s == null) return {};
0138     try {
0139         return (jsonDecode(s) as Map).cast<String, dynamic>();
0140     } catch (_) {
0141         return {};
0142     }
0143 }
0145 static Future<void> setCurrentNovelId(String? id) async {
0146     if (id == null) {
0147         await _p.remove(_kCurrentNovelId);
0148     } else {
0149         await _p.setString(_kCurrentNovelId, id);
0150     }
0151 }
0153 static String? getCurrentNovelId() => _p.getString(_kCurrentNovelId);
0155 static int getCurrentNovelIndex() => _p.getInt(_kNovelIndex) ?? 0;

```

```
0157 static int getCurrentChapterIndex() => _p.getInt(_kCurrentChapterIndex) ?? 0;
0159 static Future<void> setCurrentChapterIndex(int index) async {
0160     await _p.setInt(_kCurrentChapterIndex, index);
0161 }
0163 static Future<void> setCurrentNovelIndex(int index) async {
0164     await _p.setInt(_kNovelIndex, index);
0165 }
0167 // —— 书籍正文内容（替代 IndexedDB LocalDB.js） ——
0168 static Future<File> _bookFile(String bookId) async {
0169     final dir = await getApplicationDocumentsDirectory();
0170     return File('${dir.path}/books/$bookId.json');
0171 }
0173 static Future<void> saveBookContent(
0174     String bookId, {
0175     required List<String> lines,
0176     required List<Map<String, dynamic>> chapters,
0177 }) async {
0178     try {
0179         final file = await _bookFile(bookId);
0180         await file.parent.create(recursive: true);
0181         await file
0182             .writeAsString(jsonEncode({'lines': lines, 'chapters': chapters}));
0183     } catch (e, st) {
0184         CyberLogger.captureWarning(
0185             e,
0186             stack: st,
0187             tag: 'library',
0188             extra: {'context': 'StorageService.saveBookContent 失败'},
0189         );
0190     }
0191 }
0193 static Future<Map<String, dynamic>?> loadBookContent(String bookId) async {
0194     try {
0195         final file = await _bookFile(bookId);
0196         if (!await file.exists()) return null;
0197         final content = await file.readAsString();
0198         if (content.trim().isEmpty) return null;
0199         return jsonDecode(content) as Map<String, dynamic>;
0200     } catch (e, st) {
0201         CyberLogger.captureWarning(
0202             e,
0203             stack: st,
0204             tag: 'library',
0205             extra: {'context': 'StorageService.loadBookContent 失败'},
0206         );
0207         return null;
0208     }
0209 }
0211 static Future<void> deleteBookContent(String bookId) async {
0212     try {
```

```

0213     final file = await _bookFile(bookId);
0214     if (await file.exists()) await file.delete();
0215   } catch (_) {}
0216 }
0218 // —— 全局设置 ——
0219 static bool getSettingSound() => _p.getBool(_kSettingSound) ?? true;
0220 static Future<void> setSettingSound(bool v) => _p.setBool(_kSettingSound, v);
0222 static bool getSettingStoryTts() => _p.getBool(_kSettingStoryTts) ?? true;
0223 static Future<void> setSettingStoryTts(bool v) =>
0224     _p.setBool(_kSettingStoryTts, v);
0226 static String getSettingVoice() =>
0227     _p.getString(_kSettingVoice) ?? 'zh-CN-XiaoxiaoNeural';
0228 static Future<void> setSettingVoice(String v) =>
0229     _p.setString(_kSettingVoice, v);
0231 static int getSettingIdleTimeout() => _p.getInt(_kSettingIdleTimeout) ?? 1;
0232 static Future<void> setSettingIdleTimeout(int v) =>
0233     _p.setInt(_kSettingIdleTimeout, v);
0235 static double getSettingTtsRate() => _p.getDouble(_kSettingTtsRate) ?? 1.0;
0236 static Future<void> setSettingTtsRate(double v) =>
0237     _p.setDouble(_kSettingTtsRate, v);
0239 static double getSettingAmbientVol() =>
0240     _p.getDouble(_kSettingAmbientVol) ?? 0.5;
0241 static Future<void> setSettingAmbientVol(double v) =>
0242     _p.setDouble(_kSettingAmbientVol, v);
0244 static bool getSettingAmbientEnabled() =>
0245     _p.getBool(_kSettingAmbientEnabled) ?? true;
0246 static Future<void> setSettingAmbientEnabled(bool v) =>
0247     _p.setBool(_kSettingAmbientEnabled, v);
0249 static String getSettingAmbientStyle() =>
0250     _p.getString(_kSettingAmbientStyle) ?? 'wuxia';
0251 static Future<void> setSettingAmbientStyle(String v) =>
0252     _p.setString(_kSettingAmbientStyle, v);
0254 static String getSettingAnimationQuality() =>
0255     _p.getString(_kSettingAnimationQuality) ?? 'auto';
0256 static Future<void> setSettingAnimationQuality(String v) =>
0257     _p.setString(_kSettingAnimationQuality, v);
0259 // —— 隐私协议同意状态 ——
0260 static bool hasAgreedPrivacy() => _p.getBool(_kHasAgreedPrivacy) ?? false;
0261 static Future<void> setHasAgreedPrivacy(bool v) =>
0262     _p.setBool(_kHasAgreedPrivacy, v);
0264 // —— 粘性位：用户是否曾主动选择过书籍 ——
0265 static bool hasSelectedBook() => _p.getBool(_kHasSelectedBook) ?? false;
0266 static Future<void> setHasSelectedBook(bool v) =>
0267     _p.setBool(_kHasSelectedBook, v);
0269 // —— 分章文本缓存（文件路径，非 SharedPreferences） ——
0270 static Future<File> _chapterCacheFile(String bookId, int chapterIndex) async {
0271     final dir = await getApplicationDocumentsDirectory();
0272     final idx = chapterIndex.toString().padLeft(3, '0');
0273     return File('${dir.path}/books/chapters/$bookId/$idx.txt');
0274 }

```

```
0276 static Future<Directory> _chapterCacheDir(String bookId) async {
0277     final dir = await getApplicationDocumentsDirectory();
0278     return Directory('${dir.path}/books/chapters/$bookId');
0279 }
0281 static Future<void> saveChapterCache(
0282     String bookId,
0283     int chapterIndex,
0284     String text,
0285 ) async {
0286     try {
0287         final file = await _chapterCacheFile(bookId, chapterIndex);
0288         await file.parent.create(recursive: true);
0289         await file.writeAsString(text);
0290     } catch (e, st) {
0291         CyberLogger.captureWarning(
0292             e,
0293             stack: st,
0294             tag: 'library',
0295             extra: {'context': 'StorageService.saveChapterCache 失败'},
0296         );
0297     }
0298 }
0300 static Future<String?> loadChapterCache(
0301     String bookId,
0302     int chapterIndex,
0303 ) async {
0304     try {
0305         final file = await _chapterCacheFile(bookId, chapterIndex);
0306         if (!await file.exists()) return null;
0307         final content = await file.readAsString();
0308         return content.trim().isEmpty ? null : content;
0309     } catch (e, st) {
0310         CyberLogger.captureWarning(
0311             e,
0312             stack: st,
0313             tag: 'library',
0314             extra: {'context': 'StorageService.loadChapterCache 失败'},
0315         );
0316         return null;
0317     }
0318 }
0320 static Future<void> clearChapterCache(String bookId) async {
0321     try {
0322         final dir = await _chapterCacheDir(bookId);
0323         if (await dir.exists()) await dir.delete(recursive: true);
0324     } catch (e, st) {
0325         CyberLogger.captureWarning(
0326             e,
0327             stack: st,
0328             tag: 'library',
```

```
0329         extra: {'context': 'StorageService.clearChapterCache 失败'},
0330     );
0331 }
0332 }
0334 /// LRU 清理: 保留 [currentChapterIndex ± keepAround] 范围内文件, 其余删除。
0335 static Future<void> pruneChapterCache(
0336     String bookId,
0337     int currentChapterIndex, {
0338     int keepAround = 3,
0339 }) async {
0340     try {
0341         final dir = await _chapterCacheDir(bookId);
0342         if (!await dir.exists()) return;
0343         final keepMin = currentChapterIndex - keepAround;
0344         final keepMax = currentChapterIndex + keepAround;
0345         await for (final entity in dir.list()) {
0346             if (entity is! File) continue;
0347             final name = entity.uri.pathSegments.last;
0348             if (!name.endsWith('.txt')) continue;
0349             final idx = int.tryParse(name.replaceFirst('.txt', ''));
0350             if (idx == null) continue;
0351             if (idx < keepMin || idx > keepMax) await entity.delete();
0352         }
0353     } catch (e, st) {
0354         CyberLogger.captureWarning(
0355             e,
0356             stack: st,
0357             tag: 'library',
0358             extra: {'context': 'StorageService.pruneChapterCache 失败'},
0359         );
0360     }
0361 }
0363 // —— 书目录缓存 (文件路径) ——
0364 static Future<File> _catalogFile(String bookId) async {
0365     final dir = await getApplicationDocumentsDirectory();
0366     return File('${dir.path}/books/catalogs/${bookId}_catalog.json');
0367 }
0369 static Future<void> saveBookCatalog(
0370     String bookId,
0371     List<Map<String, dynamic>> chapters,
0372 ) async {
0373     try {
0374         final file = await _catalogFile(bookId);
0375         await file.parent.create(recursive: true);
0376         await file.writeAsString(jsonEncode(chapters));
0377     } catch (e, st) {
0378         CyberLogger.captureWarning(
0379             e,
0380             stack: st,
0381             tag: 'library',
```

```

0382         extra: {'context': 'StorageService.saveBookCatalog 失败'},
0383     );
0384 }
0385 }
0387 static Future<List<Map<String, dynamic>>>> loadBookCatalog(
0388     String bookId,
0389 ) async {
0390     try {
0391         final file = await _catalogFile(bookId);
0392         if (!await file.exists()) return null;
0393         final content = await file.readAsString();
0394         if (content.trim().isEmpty) return null;
0395         return (jsonDecode(content) as List).cast<Map<String, dynamic>>();
0396     } catch (e, st) {
0397         CyberLogger.captureWarning(
0398             e,
0399             stack: st,
0400             tag: 'library',
0401             extra: {'context': 'StorageService.loadBookCatalog 失败'},
0402         );
0403         return null;
0404     }
0405 }
0406 }

```

// ===== 文件: lib/core/theme/cyber_colors.dart =====

```

0001 import 'package:flutter/material.dart';
0003 class CyberColors {
0004     // 极致的深色背景
0005     static const Color background = Color(0xFF000000); // 真正的高级黑
0007     // 赛博/黑客帝国经典的荧光绿（主色调）
0008     static const Color neonGreen = Color(0xFF00FF41);
0010     // 赛博朋克必不可少的霓虹粉
0011     static const Color neonPink = Color(0xFFFFE019A);
0013     // 霓虹紫色
0014     static const Color neonPurple = Color(0xFF8B5CF6);
0016     // 霓虹青色
0017     static const Color neonCyan = Color(0xFF22D3EE);
0019     // 卡片背景色（带一点透明度的深灰，制造玻璃感）
0020     static const Color cardBackground = Color(0xFF13141E);
0021     static const Color surface = Color(0xFF1A1B28);
0023     // 提词器未读文字的暗色
0024     static const Color textDim = Color(0xFF4A5568);
0026     // 毛玻璃深底色（灵动岛 / 顶部工具栏 / 弹窗共用）
0027     static const Color glassDark = Color(0xD90A0A0F);
0029     // 播放按钮渐变粉（CyberPlayerConsole 播放按钮起始色）
0030     static const Color hotPink = Color(0xFFE04899);
0032     // 章节列表 / 弹窗背景
0033     static const Color panelBackground = Color(0xFF0D0E18);
0035     // 白色语义化透明度

```

```
0036 static const Color whiteHigh = Color(0xD9FFFFFF); // 85%
0037 static const Color whiteMedium = Color(0x99FFFFFF); // 60%
0038 static const Color whiteDim = Color(0x8AFFFFFF); // 54%
0039 static const Color whiteMuted = Color(0x61FFFFFF); // 38%
0040 static const Color whiteSubtle = Color(0x3DFFFFFF); // 24%
0041 static const Color whiteFaint = Color(0x14FFFFFF); // 8%
0042 static const Color whiteBorder = Color(0x1AFFFFFF); // 10%
0044 // 黑色语义化透明度（阴影 / 遮罩 / overlay）
0045 static const Color blackOverlay = Color(0xB3000000); // 70%
0046 static const Color blackShadow = Color(0x80000000); // 50%
0047 static const Color blackDim = Color(0x8A000000); // 54%
0049 // 柔和的发光阴影色
0050 static const Color glowShadow = Color(0x8800FF41);
0051 static const Color pinkGlow = Color(0x88FE019A);
0053 // 2048 方块配色 - 提取自旧版style.css的渐变色
0054 static const Color tile2Start = Color(0xFF43e97b);
0055 static const Color tile2End = Color(0xFF38f9d7);
0057 static const Color tile4Start = Color(0xFF00f2fe);
0058 static const Color tile4End = Color(0xFF4facfe);
0060 static const Color tile8Start = Color(0xFF4facfe);
0061 static const Color tile8End = Color(0xFF00c6fb);
0063 static const Color tile16Start = Color(0xFF0093e9);
0064 static const Color tile16End = Color(0xFF80d0c7);
0066 static const Color tile32Start = Color(0xFFa18cd1);
0067 static const Color tile32End = Color(0xFFfbc2eb);
0069 static const Color tile64Start = Color(0xFFc471f5);
0070 static const Color tile64End = Color(0xFFfa71cd);
0072 static const Color tile128Start = Color(0xFFe040fb);
0073 static const Color tile128End = Color(0xFF8e24aa);
0075 static const Color tile256Start = Color(0xFFf857a6);
0076 static const Color tile256End = Color(0xFFff5858);
0078 static const Color tile512Start = Color(0xFFf7971e);
0079 static const Color tile512End = Color(0xFFffd200);
0081 static const Color tile1024Start = Color(0xFFff6a00);
0082 static const Color tile1024End = Color(0xFFee0979);
0084 static const Color tile2048Start = Color(0xFFf12711);
0085 static const Color tile2048End = Color(0xFFf5af19); // 亮红
0086 static const Color tile512 = Color(0xFFFFFD60A); // 金色
0088 // 工具色
0089 static const Color white = Color(0xFFFFFFFF); // 纯白（Canvas 高光专用）
0090 static const Color transparent = Color(0x00000000); // 透明（Material 背景专用）
0091 static const Color tileGold = Color(0xFFFFFD700); // 传奇金色（square_board 溯源）
0092 static const Color hackerBlue =
0093     Color(0xFF00F3FF); // 骇客蓝（规范中的 #00f3ff，吹着天 BGM）
0094 static const Color tileBlue = Color(0xFF3B82F6); // 方块低数粒子色（tile_widget 溯源）
0095 }
```

// ===== 文件：lib/core/theme/cyber_dimensions.dart =====

```
0001 /// 赛博朋克设计系统 - 尺寸规范
0002 /// 统一管理圆角、边框、模糊、间距等数值，确保视觉一致性
```



```
0003 class CyberDimensions {
0004     // ===== 圆角系统 =====
0005     // 5 级圆角，从大到小递减
0007     /// 超大圆角 - 用于大型容器（棋盘外框）
0008     static const double radiusXL = 32.0;
0010     /// 大圆角 - 用于中型容器（控制台、工具栏、弹窗）
0011     static const double radiusL = 24.0;
0013     /// 中圆角 - 用于卡片、格子
0014     static const double radiusM = 16.0;
0016     /// 小圆角 - 用于按钮、小卡片
0017     static const double radiusS = 12.0;
0019     /// 超小圆角 - 用于细节（进度条、波形条）
0020     static const double radiusXS = 2.0;
0022     // ===== 边框系统 =====
0023     // 3 级边框宽度
0025     /// 粗边框 - 用于强调（选中状态、主要边框）
0026     static const double borderThick = 1.5;
0028     /// 常规边框 - 用于普通容器
0029     static const double borderNormal = 1.0;
0031     /// 细边框 - 用于细微装饰（背景网格）
0032     static const double borderThin = 0.5;
0034     // ===== 毛玻璃模糊系统 =====
0035     // 3 级模糊强度
0037     /// 强模糊 - 用于主要容器（棋盘、卡片）
0038     static const double blurStrong = 20.0;
0040     /// 中模糊 - 用于工具栏、控制台
0041     static const double blurMedium = 15.0;
0043     /// 轻模糊 - 用于遮罩、头部
0044     static const double blurLight = 10.0;
0046     // ===== 间距系统 =====
0047     // 标准间距，8px 倍数
0049     static const double spacingXXS = 2.0;
0050     static const double spacingXS = 4.0;
0051     static const double spacingS = 8.0;
0052     static const double spacingMS = 12.0;
0053     static const double spacingM = 16.0;
0054     static const double spacingML = 20.0;
0055     static const double spacingL = 24.0;
0056     static const double spacingXL = 32.0;
0058     // ===== 头部 / 工具栏 =====
0060     /// 标准页面头部高度
0061     static const double headerHeight = 56.0;
0063     /// 标准电传屏高度
0064     static const double teleprompterHeight = 46.0;
0066     /// 标准电传屏遮罩宽度
0067     static const double teleprompterMaskWidth = 40.0;
0069     /// 标准仪表盘吉祥物宽度
0070     static const double dashboardMascotWidth = 68.0;
0072     /// 标准仪表盘吉祥物高度
0073     static const double dashboardMascotHeight = 84.0;
```

```

0075  /// 标准仪表盘缓冲区高度
0076  static const double dashboardBoardBuffer = 76.0;
0078  /// 标准仪表盘状态卡片最小高度
0079  static const double dashboardStatusCardMinHeight = 85.0;
0081  // ===== 动效时长 =====
0083  /// 通用过渡动画时长（Modal、开关、按钮状态切换）
0084  static const Duration animNormal = Duration(milliseconds: 300);
0086  /// 快速反馈动画时长（弹窗退场、选项切换）
0087  static const Duration animFast = Duration(milliseconds: 250);
0089  /// 超快反馈动画时长（棋盘倾斜、元素位移）
0090  static const Duration animXFast = Duration(milliseconds: 150);
0092  /// 瞬间反馈动画时长（眼球追踪、合并弹出）
0093  static const Duration animInstant = Duration(milliseconds: 120);
0095  /// 中等动画时长（合并粒子、导入按钮）
0096  static const Duration animMedium = Duration(milliseconds: 400);
0098  /// 消除动画时长（棋盘方块消除三段式）
0099  static const Duration animEliminate = Duration(milliseconds: 450);
0101  /// 慢动画时长（棋盘重置、身体跳跃）
0102  static const Duration animSlow = Duration(milliseconds: 600);
0104  /// Toast 显示持续时长
0105  static const Duration toastDuration = Duration(seconds: 2, milliseconds: 500);
0107  // ===== 动效（视觉） =====
0109  /// 提词器扫光条宽度
0110  static const double shimmerWidth = 80.0;
0112  /// 发光阴影模糊半径（霓虹光晕）
0113  static const double glowBlurRadius = 8.0;
0115  /// 发光阴影扩散半径
0116  static const double glowSpreadRadius = 1.0;
0118  // ===== 列表 / 网格 =====
0120  /// 章节列表 item 固定高度（复用 headerHeight）
0121  static const double chapterItemHeight = headerHeight;
0123  /// 状态圆点尺寸（已读/未读指示器）
0124  static const double statusDotSize = 6.0;
0126  // ===== 图标尺寸 =====
0128  /// 超小图标（箭头、指示器）
0129  static const double iconXS = 16.0;
0131  /// 小图标（音量、状态）
0132  static const double iconS = 18.0;
0134  /// 中图标（操作按钮、加载器）
0135  static const double iconM = 20.0;
0137  /// 大图标（对话框标题图标）
0138  static const double iconL = 28.0;
0139 }

// ===== 文件：lib/core/theme/cyber_shadows.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'cyber_colors.dart';
0004 /// 赛博朋克设计系统 - 阴影规范
0005 /// 统一管理阴影效果，确保视觉层次一致
0006 class CyberShadows {

```

```
0007 // ===== 标准阴影系统 =====
0009 /// 悬浮阴影 - 用于大型容器（棋盘、卡片）
0010 /// 强烈的悬浮感，适合主要内容区域
0011 static const List<BoxShadow> elevated = [
0012     BoxShadow(
0013         color: CyberColors.blackShadow,
0014         blurRadius: 30,
0015         offset: Offset(0, 10),
0016     ),
0017 ];
0019 /// 浮动阴影 - 用于中型组件（书架卡片、导入按钮）
0020 /// 轻微悬浮感，适合次要内容
0021 static const List<BoxShadow> floating = [
0022     BoxShadow(
0023         color: CyberColors.blackShadow,
0024         blurRadius: 16,
0025         offset: Offset(0, 6),
0026     ),
0027 ];
0029 /// 贴近阴影 - 用于小型组件（播放按钮、小卡片）
0030 /// 贴近表面，适合细节装饰
0031 static const List<BoxShadow> subtle = [
0032     BoxShadow(
0033         color: CyberColors.blackShadow,
0034         blurRadius: 10,
0035         offset: Offset(0, 4),
0036     ),
0037 ];
0039 // ===== 霓虹光晕阴影 =====
0041 /// 创建霓虹光晕效果
0042 /// 用于方块、按钮等需要发光效果的元素
0043 static List<BoxShadow> neonGlow({
0044     required Color color,
0045     double intensity = 1.0,
0046 }) {
0047     return [
0048         BoxShadow(
0049             color: color.withValues(alpha: 0.6 * intensity),
0050             blurRadius: 12 * intensity,
0051             spreadRadius: 2 * intensity,
0052         ),
0053         BoxShadow(
0054             color: color.withValues(alpha: 0.3 * intensity),
0055             blurRadius: 18 * intensity,
0056             spreadRadius: 0,
0057         ),
0058     ];
0059 }
0060 }
```

```
// ===== 文件: lib/core/theme/cyber_text_styles.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'cyber_colors.dart';
0004 class CyberTextStyles {
0005   static const String monoFont = 'JetBrains Mono';
0006   // 提词器正在阅读的【高亮发光文字】
0007   static const TextStyle teleprompterActive = TextStyle(
0008     color: CyberColors.neonGreen,
0009     fontSize: 24,
0010     fontWeight: FontWeight.bold,
0011     letterSpacing: 2.0, // 字间距拉开, 更有极客敲代码的呼吸感
0012     shadows: [
0013       Shadow(
0014         blurRadius: 10.0, // 发光半径
0015         color: CyberColors.glowShadow,
0016         offset: Offset(0, 0),
0017       ),
0018     ],
0019   );
0021   // 提词器下方还没读到的【暗色背景文字】
0022   static const TextStyle teleprompterDim = TextStyle(
0023     color: CyberColors.textDim,
0024     fontSize: 24,
0025     fontWeight: FontWeight.normal,
0026     letterSpacing: 2.0,
0027   );
0029   // 2048 棋盘上的数字字体
0030   static const TextStyle gameGridNumber = TextStyle(
0031     color: CyberColors.background,
0032     fontSize: 28,
0033     fontWeight: FontWeight.w900,
0034   );
0036   // ===== 通用 UI 文本样式 =====
0038   /// 页面头部标题 (18px, bold, letterSpacing: 2)
0039   /// 使用时通过 copyWith(color: ...) 设置具体颜色
0040   static const TextStyle screenTitle = TextStyle(
0041     fontSize: 18,
0042     fontWeight: FontWeight.bold,
0043     letterSpacing: 2,
0044   );
0046   /// 分区标题 (12px, bold, letterSpacing: 1.5, neonGreen)
0047   static const TextStyle sectionLabel = TextStyle(
0048     color: CyberColors.neonGreen,
0049     fontSize: 12,
0050     fontWeight: FontWeight.bold,
0051     letterSpacing: 1.5,
0052   );
0054   /// 列表项主标题 (14px, whiteHigh)
0055   static const TextStyle tileTitle = TextStyle(
0056     color: CyberColors.whiteHigh,
```

```
0057     fontSize: 14,
0058 );
0060 /// 列表项副标题 (12px, whiteMuted)
0061 static const TextStyle tileSubtitle = TextStyle(
0062     color: CyberColors.whiteMuted,
0063     fontSize: 12,
0064 );
0066 /// 标签文字 (14px, whiteDim)
0067 static const TextStyle labelMedium = TextStyle(
0068     color: CyberColors.whiteDim,
0069     fontSize: 14,
0070 );
0072 /// 小号正文 (13px, whiteDim) —— 下拉菜单、标签芯片等
0073 static const TextStyle bodySmall = TextStyle(
0074     color: CyberColors.whiteDim,
0075     fontSize: 13,
0076 );
0078 /// 小号正文加粗 (13px, w600, whiteHigh)
0079 static const TextStyle bodySmallBold = TextStyle(
0080     color: CyberColors.whiteHigh,
0081     fontSize: 13,
0082     fontWeight: FontWeight.w600,
0083 );
0085 static const TextStyle overlineTiny = TextStyle(
0086     color: CyberColors.whiteDim,
0087     fontSize: 9,
0088     fontWeight: FontWeight.w700,
0089     letterSpacing: 0.5,
0090 );
0092 static const TextStyle segmentLabel = TextStyle(
0093     color: CyberColors.whiteHigh,
0094     fontSize: 14,
0095     fontWeight: FontWeight.w600,
0096     letterSpacing: 0.3,
0097 );
0099 static const TextStyle teleprompterInlineRead = TextStyle(
0100     color: CyberColors.neonCyan,
0101     fontSize: 18,
0102     fontWeight: FontWeight.w800,
0103     letterSpacing: 0.5,
0104     height: 1.0,
0105 );
0107 static const TextStyle teleprompterInlineUnread = TextStyle(
0108     color: CyberColors.whiteMuted,
0109     fontSize: 18,
0110     fontWeight: FontWeight.w500,
0111     letterSpacing: 0.5,
0112     height: 1.0,
0113 );
0115 static const TextStyle teleprompterError = TextStyle(
```

```
0116     color: CyberColors.whiteHigh,
0117     fontSize: 12,
0118     fontWeight: FontWeight.w700,
0119 );
0121 /// 提词器浮层小号错误标题 (10px bold)
0122 static const TextStyle teleprompterErrorTitle = TextStyle(
0123     fontSize: 10,
0124     fontWeight: FontWeight.bold,
0125 );
0127 /// 提词器浮层小号提示文字 (9px)
0128 static const TextStyle teleprompterErrorHint = TextStyle(
0129     fontSize: 9,
0130 );
0132 static const TextStyle teleprompterPlaceholder = TextStyle(
0133     color: CyberColors.whiteMuted,
0134     fontSize: 14,
0135     fontWeight: FontWeight.w600,
0136 );
0138 static const TextStyle dashboardCounter = TextStyle(
0139     color: CyberColors.neonCyan,
0140     fontSize: 22,
0141     fontFamily: monoFont,
0142     fontWeight: FontWeight.w900,
0143 );
0145 static const TextStyle dashboardSeparator = TextStyle(
0146     color: CyberColors.whiteSubtle,
0147     fontSize: 16,
0148     fontFamily: monoFont,
0149     fontWeight: FontWeight.w300,
0150 );
0152 static const TextStyle captionBold = TextStyle(
0153     color: CyberColors.whiteDim,
0154     fontSize: 12,
0155     fontWeight: FontWeight.bold,
0156 );
0158 static const TextStyle captionTight = TextStyle(
0159     color: CyberColors.whiteDim,
0160     fontSize: 12,
0161     height: 1.3,
0162 );
0164 static const TextStyle captionComfortable = TextStyle(
0165     color: CyberColors.whiteDim,
0166     fontSize: 12,
0167     height: 1.4,
0168 );
0170 static const TextStyle captionHint = TextStyle(
0171     color: CyberColors.whiteDim,
0172     fontSize: 11,
0173     height: 1.5,
0174 );
```

```

0658     const Icon(
0659       Icons.volume_down,
0660       size: 14,
0661       color: CyberColors.whiteMuted,
0662     ),
0663     Expanded(
0664       child: SliderTheme(
0665         data: SliderTheme.of(context).copyWith(
0666           activeTrackColor: CyberColors.neonPurple,
0667           inactiveTrackColor: CyberColors.whiteFaint,
0668           thumbColor: CyberColors.white,
0669           overlayColor: CyberColors.neonPurple.withValues(alpha: 0.1),
0670           trackHeight: 2.0,
0671           thumbShape: const RoundSliderThumbShape(
0672             enabledThumbRadius: 6,
0673           ),
0674         ),
0675         child: Slider(
0676           value: settings.ambientVol,
0677           onChanged: (v) async {
0678             await settings.setAmbientVol(v);
0679             await AmbientService.setVolume(v);
0680           },
0681         ),
0682       ),
0683     ),
0684     Text(
0685       '${(settings.ambientVol * 100).round()}%',
0686       style: CyberTextStyles.captionBold.copyWith(
0687         color: CyberColors.neonPurple,
0688         fontSize: 10,
0689         fontFamily: CyberTextStyles.monoFont,
0690       ),
0691     ),
0692     const SizedBox(width: 12),
0693   ],
0694 ),
0695 );
0696 }
0697 }

// ===== 文件: lib/features/settings/presentation/widgets/privacy_agreement_modal.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'package:flutter/services.dart';
0003 import 'package:url_launcher/url_launcher.dart';
0004 import 'package:yueyou/core/config/app_info_config.dart';
0005 import 'package:yueyou/core/theme/cyber_colors.dart';
0006 import 'package:yueyou/core/theme/cyber_dimensions.dart';
0007 import 'package:yueyou/core/theme/cyber_text_styles.dart';
0008 import 'package:yueyou/shared/widgets/cyber_modal.dart';

```

```

0010 /// 赛博朋克风格隐私协议弹窗
0011 /// - barrierDismissible: false, 强制用户作出选择
0012 /// - 返回 true 表示同意, 用户拒绝时直接退出应用
0013 Future<bool> showPrivacyAgreementModal (BuildContext context) async {
0014   final result = await showCyberModal<bool>(
0015     context: context,
0016     barrierDismissible: false,
0017     child: const _PrivacyAgreementContent(),
0018   );
0019   return result ?? false;
0020 }
0021
0022 class _PrivacyAgreementContent extends StatelessWidget {
0023   const _PrivacyAgreementContent();
0024
0025   @override
0026   Widget build(BuildContext context) {
0027     return Padding(
0028       padding: const EdgeInsets.all(CyberDimensions.spacingL),
0029       child: Column(
0030         mainAxisAlignment: MainAxisAlignment.min,
0031         children: [
0032           // —— 图标 + 标题 ——
0033           const Icon(
0034             Icons.security_rounded,
0035             color: CyberColors.neonCyan,
0036             size: CyberDimensions.iconL,
0037           ),
0038           const SizedBox(height: CyberDimensions.spacingS),
0039           Text(
0040             '阅游 · 隐私政策',
0041             style: CyberTextStyles.dialogTitle.copyWith(
0042               color: CyberColors.neonCyan,
0043             ),
0044           ),
0045           const SizedBox(height: CyberDimensions.spacingXS),
0046           Text(
0047             '神经接驳协议 · NEURAL LINK v1.0',
0048             style: CyberTextStyles.caption.copyWith(
0049               color: CyberColors.whiteMuted,
0050               letterSpacing: 1.5,
0051             ),
0052           ),
0053           const SizedBox(height: CyberDimensions.spacingM),
0054           // —— 协议正文 (可滚动) ——
0055           Container(
0056             constraints: BoxConstraints(
0057               maxHeight: MediaQuery.sizeOf(context).height * 0.4,
0058             ),
0059           ),
0060           decoration: BoxDecoration(
0061             color: CyberColors.whiteFaint,
0062             borderRadius: BorderRadius.circular(CyberDimensions.radiusS),

```



```

0063         border: Border.all(
0064             color: CyberColors.neonCyan.withValues(alpha: 0.2),
0065             width: CyberDimensions.borderThin,
0066         ),
0067     ),
0068     child: const SingleChildScrollView(
0069         padding: EdgeInsets.all(CyberDimensions.spacingM),
0070         child: Column(
0071             crossAxisAlignment: CrossAxisAlignment.start,
0072             children: [
0073                 _PolicySection(
0074                     icon: '',
0075                     title: '数据存储',
0076                     body: '所有阅读进度、游戏数据及用户设置仅存储于您的本地设备。'
0077                         '我们不会将您的个人数据上传至任何远端服务器。',
0078                 ),
0079                 SizedBox(height: CyberDimensions.spacingM),
0080                 _PolicySection(
0081                     icon: '',
0082                     title: '云端 TTS 合成',
0083                     body: '当您启用听书功能时，当前朗读的文本段落将通过加密信道发送至'
0084                         '云端 TTS 服务以合成音频。除朗读文本外，不传输任何其他信息。',
0085                 ),
0086                 SizedBox(height: CyberDimensions.spacingM),
0087                 _PolicySection(
0088                     icon: '',
0089                     title: '存储权限',
0090                     body: '导入数据芯片（TXT 文件）时，本应用需要访问您设备的本地存储。'
0091                         '此权限仅用于读取您主动选择的文件，不会扫描其他目录。',
0092                 ),
0093                 SizedBox(height: CyberDimensions.spacingM),
0094                 _PolicySection(
0095                     icon: '',
0096                     title: '隐私承诺',
0097                     body: '- 不采集设备标识符或位置信息\n'
0098                         '- 不追踪用户行为或使用习惯\n'
0099                         '- 不向任何第三方共享数据\n'
0100                         '- 符合《个人信息保护法》及 GDPR',
0101                 ),
0102                 SizedBox(height: CyberDimensions.spacingM),
0103                 _PolicySection(
0104                     icon: '',
0105                     title: '开发者信息',
0106                     body: '运营主体: ${AppInfoConfig.developerName}\n'
0107                         '联系邮箱: ${AppInfoConfig.contactEmail}\n'
0108                         '隐私问题反馈、数据删除请求请发送邮件至上述邮箱。',
0109                 ),
0110             ],
0111         ),
0112     ),

```

```
0113     ),
0114     const SizedBox(height: CyberDimensions.spacingM),
0115     // —— 提示 + 隐私政策外链 ——
0116     Wrap(
0117       alignment: WrapAlignment.center,
0118       crossAxisAlignment: WrapCrossAlignment.center,
0119       children: [
0120         Text(
0121           '同意后, 您可在设置页面随时重新查看本协议 ',
0122           textAlign: TextAlign.center,
0123           style: CyberTextStyles.caption.copyWith(
0124             color: CyberColors.whiteMuted,
0125           ),
0126         ),
0127       ],
0128       GestureDetector(
0129         onTap: () => launchUrl(
0130           Uri.parse(AppInfoConfig.privacyPolicyUrl),
0131           mode: LaunchMode.externalApplication,
0132         ),
0133         child: Text(
0134           '阅读完整版《阅游隐私政策》',
0135           style: CyberTextStyles.caption.copyWith(
0136             color: CyberColors.neonCyan,
0137             decoration: TextDecoration.underline,
0138             decorationColor: CyberColors.neonCyan,
0139           ),
0140         ),
0141       ),
0142     ],
0143   ),
0144   const SizedBox(height: CyberDimensions.spacingL),
0145   // —— 按钮行 ——
0146   Row(
0147     children: [
0148       // 不同意并退出 (危险操作, 用粉色/红色边框区分)
0149       Expanded(
0150         child: _DeclineButton(
0151           onTap: () {
0152             Navigator.of(context).pop(false);
0153             SystemNavigator.pop();
0154           },
0155         ),
0156       ),
0157       const SizedBox(width: CyberDimensions.spacingMS),
0158       // 同意 (主按钮)
0159       Expanded(
0160         child: _AgreeButton(
0161           onTap: () => Navigator.of(context).pop(true),
0162         ),
0163       ),
0164     ],
```

```
0165     ],
0166   ),
0167 ],
0168 ),
0169 );
0170 }
0171 }
0173 // —— 协议条款单节 ——
0174 class _PolicySection extends StatelessWidget {
0175   final String icon;
0176   final String title;
0177   final String body;
0178   const _PolicySection({
0179     required this.icon,
0180     required this.title,
0181     required this.body,
0182   });
0183   @override
0184   Widget build(BuildContext context) {
0185     return Column(
0186       crossAxisAlignment: CrossAxisAlignment.start,
0187       children: [
0188         Row(
0189           children: [
0190             Text(
0191               icon,
0192               style: CyberTextStyles.bodySmall.copyWith(
0193                 fontSize: 14,
0194               ),
0195             ),
0196             const SizedBox(width: CyberDimensions.spacingXS),
0197             Text(
0198               title,
0199               style: CyberTextStyles.bodySmallBold.copyWith(
0200                 color: CyberColors.neonCyan,
0201               ),
0202             ),
0203           ],
0204         ),
0205         const SizedBox(height: CyberDimensions.spacingXS),
0206         Text(
0207           body,
0208           style: CyberTextStyles.captionComfortable.copyWith(
0209             color: CyberColors.whiteMedium,
0210           ),
0211         ),
0212       ],
0213     );
0214   }
0215 }
0216 }
0217 }
```

```
0219 // —— 拒绝按钮（霓虹粉边框，危险语义） ——  
0220 class _DeclineButton extends StatelessWidget {  
0221   final VoidCallback onTap;  
0222   const _DeclineButton({required this.onTap});  
0223   @override  
0224   Widget build(BuildContext context) {  
0225     return GestureDetector(  
0226       onTap: onTap,  
0227       child: Container(  
0228         padding:  
0229           const EdgeInsets.symmetric(vertical: CyberDimensions.spacingMS),  
0230         decoration: BoxDecoration(  
0231           color: CyberColors.neonPink.withValues(alpha: 0.08),  
0232           borderRadius: BorderRadius.circular(CyberDimensions.radiusS),  
0233           border: Border.all(  
0234             color: CyberColors.neonPink.withValues(alpha: 0.5),  
0235             width: CyberDimensions.borderNormal,  
0236           ),  
0237         ),  
0238         child: Center(  
0239           child: Text(  
0240             '不同意并退出',  
0241             style: CyberTextStyles.buttonLabel.copyWith(  
0242               color: CyberColors.neonPink,  
0243             ),  
0244           ),  
0245         ),  
0246       ),  
0247     );  
0248   };  
0249 }  
0250 }  
0251 // —— 同意按钮（霓虹青渐变，主语义） ——  
0252 class _AgreeButton extends StatelessWidget {  
0253   final VoidCallback onTap;  
0254   const _AgreeButton({required this.onTap});  
0255   @override  
0256   Widget build(BuildContext context) {  
0257     return GestureDetector(  
0258       onTap: onTap,  
0259       child: Container(  
0260         padding:  
0261           const EdgeInsets.symmetric(vertical: CyberDimensions.spacingMS),  
0262         decoration: BoxDecoration(  
0263           gradient: const LinearGradient(  
0264             colors: [CyberColors.neonCyan, CyberColors.neonPurple],  
0265             begin: Alignment.topLeft,  
0266             end: Alignment.bottomRight,  
0267           ),  
0268           borderRadius: BorderRadius.circular(CyberDimensions.radiusS),  
0269         ),  
0270       ),  
0271     );  
0272   };  
0273 }  
0274 }
```

```

0272         child: Center(
0273             child: Text(
0274                 '同意',
0275                 style: CyberTextStyles.buttonLabel.copyWith(
0276                     color: CyberColors.background,
0277                     fontWeight: FontWeight.w900,
0278                 ),
0279             ),
0280         ),
0281     ),
0282 );
0283 }
0284 }

// ===== 文件: lib/features/settings/providers/settings_provider.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'package:flutter_riverpod/flutter_riverpod.dart';
0004 import '../core/database/storage_service.dart';
0005 import '../core/utils/cyber_performance_detector.dart';
0006 import '../audio/services/ambient_service.dart';
0008 /// 供 Riverpod 使用的全局设置 Provider
0009 final settingsProvider = ChangeNotifierProvider<SettingsProvider>((ref) {
0010     final p = SettingsProvider();
0011     p.loadFromStorage();
0012     return p;
0013 });
0015 /// 全局设置 Provider
0016 /// 完整复刻旧版 main.js 的 `t` 设置对象 + localStorage 持久化逻辑
0017 /// 键名与 JS setting_* 前缀完全对齐
0018 class SettingsProvider with ChangeNotifier {
0019     late bool sound;
0020     late bool storyTts;
0021     late String voice;
0022     late int idleTimeout;
0023     late double ttsRate;
0024     late double ambientVol;
0025     late String ambientStyle;
0027     /// 环境音开关 — 预留字段, 待环境音播放器功能实现后启用
0028     /// 当前无 UI 入口, 也无消费方, 仅持久化存档保持向后兼容
0029     late bool ambientEnabled;
0031     /// 动画质量设置: 'auto' | 'high' | 'medium' | 'low'
0032     late String animationQualitySetting;
0033     CyberAnimationLevel? _autoDetectedLevel;
0035     /// App 启动时从 StorageService 恢复所有设置
0036     void loadFromStorage() {
0037         sound = StorageService.getSettingSound();
0038         storyTts = StorageService.getSettingStoryTts();
0039         voice = StorageService.getSettingVoice();
0040         // 校验音色有效性, 无效则回退默认 (防止 storage 残留无效音色名)
0041         if (voice != 'zh-CN-XiaoxiaoNeural' &&

```

```
0042         voice != 'zh-CN-YunxiNeural' &&
0043         voice != 'zh-CN-YunjianNeural' &&
0044         voice != 'zh-CN-XiaoyiNeural' &&
0045         voice != 'zh-CN-XiaomengNeural') {
0046     voice = 'zh-CN-XiaoxiaoNeural';
0047 }
0048 idleTimeout = StorageService.getSettingIdleTimeout();
0049 ttsRate = StorageService.getSettingTtsRate();
0050 ambientVol = StorageService.getSettingAmbientVol();
0051 ambientEnabled = StorageService.getSettingAmbientEnabled();
0052 ambientStyle = StorageService.getSettingAmbientStyle();
0053 AmbientService.setStyle(ambientStyle);
0054 animationQualitySetting = StorageService.getSettingAnimationQuality();
0055 if (animationQualitySetting == 'auto') {
0056     _autoDetectedLevel = CyberPerformanceDetector.detectLevel();
0057 }
0058 }
0060 notifyListeners();
0061 }
0062 /// 计算属性：获取当前界面应当采用的动画等级
0063 CyberAnimationLevel get currentAnimationLevel {
0064     switch (animationQualitySetting) {
0065         case 'high':
0066             return CyberAnimationLevel.high;
0067         case 'medium':
0068             return CyberAnimationLevel.medium;
0069         case 'low':
0070             return CyberAnimationLevel.low;
0071         case 'auto':
0072             return _autoDetectedLevel ??= CyberPerformanceDetector.detectLevel();
0073     }
0074 }
0075 }
0076 }
0077 Future<void> setSound(bool v) async {
0078     sound = v;
0079     await StorageService.setSettingSound(v);
0080     notifyListeners();
0081 }
0082 Future<void> setStoryTts(bool v) async {
0083     storyTts = v;
0084     await StorageService.setSettingStoryTts(v);
0085     notifyListeners();
0086 }
0087 Future<void> setVoice(String v) async {
0088     if (v != 'zh-CN-XiaoxiaoNeural' &&
0089         v != 'zh-CN-YunxiNeural' &&
0090         v != 'zh-CN-YunjianNeural' &&
0091         v != 'zh-CN-XiaoyiNeural' &&
0092         v != 'zh-CN-XiaomengNeural') {
0093         v = 'zh-CN-XiaoxiaoNeural';
0094     }
0095 }
```

```
0098     voice = v;
0099     await StorageService.setSettingVoice(v);
0100     notifyListeners();
0101   }
0103   Future<void> setIdleTimeout(int v) async {
0104     idleTimeout = v;
0105     await StorageService.setSettingIdleTimeout(v);
0106     notifyListeners();
0107   }
0109   Future<void> setTtsRate(double v) async {
0110     ttsRate = v;
0111     await StorageService.setSettingTtsRate(v);
0112     notifyListeners();
0113   }
0115   Future<void> setAmbientVol(double v) async {
0116     ambientVol = v;
0117     await StorageService.setSettingAmbientVol(v);
0118     notifyListeners();
0119   }
0121   Future<void> setAmbientEnabled(bool v) async {
0122     ambientEnabled = v;
0123     await StorageService.setSettingAmbientEnabled(v);
0124     notifyListeners();
0125   }
0127   Future<void> setAmbientStyle(String v) async {
0128     ambientStyle = v;
0129     await StorageService.setSettingAmbientStyle(v);
0130     notifyListeners();
0131   }
0133   Future<void> setAnimationQualitySetting(String v) async {
0134     animationQualitySetting = v;
0135     if (v == 'auto') {
0136       _autoDetectedLevel = CyberPerformanceDetector.detectLevel();
0137     }
0138     await StorageService.setSettingAnimationQuality(v);
0139     notifyListeners();
0140   }
0141 }
```

// ===== 文件: lib/features/update/domain/update_info.dart =====

```
0001 import 'package:flutter/foundation.dart';
0003 /// 服务端返回的版本信息数据类
0004 ///
0005 /// 对应 API 响应格式:
0006 /// ```json
0007 /// {
0008 ///   "version": "1.2.0",
0009 ///   "buildNumber": 3,
0010 ///   "forceUpdate": false,
0011 ///   "releaseNotes": "修复了 TTS 连接偶发断线问题",
```

```

0012 /// "downloadUrl": "https://..."
0013 /// }
0014 /// ```
0015 @immutable
0016 class UpdateInfo {
0017   /// 服务端最新版本号（语义化版本，如 "1.2.0"）
0018   final String version;
0020   /// 服务端最新构建号（整数，对应 buildNumber）
0021   final int buildNumber;
0023   /// 是否强制更新（true 时禁用跳过按钮）
0024   final bool forceUpdate;
0026   /// 版本更新说明（可选，用于展示给用户）
0027   final String releaseNotes;
0029   /// Android APK 下载链接（跳转应用市场或自定义页）
0030   final String downloadUrl;
0032   const UpdateInfo({
0033     required this.version,
0034     required this.buildNumber,
0035     required this.forceUpdate,
0036     this.releaseNotes = '',
0037     this.downloadUrl = '',
0038   });
0040   /// 从 JSON 解析（宽松策略：字段缺失时使用安全默认值，不抛异常）
0041   factory UpdateInfo.fromJson(Map<String, dynamic> json) {
0042     return UpdateInfo(
0043       version: json['version'] as String? ?? '0.0.0',
0044       buildNumber: (json['buildNumber'] as num?)?.toInt() ?? 0,
0045       forceUpdate: json['forceUpdate'] as bool? ?? false,
0046       releaseNotes: json['releaseNotes'] as String? ?? '',
0047       downloadUrl: json['downloadUrl'] as String? ?? '',
0048     );
0049   }
0051   @override
0052   String toString() =>
0053     'UpdateInfo(version=$version, buildNumber=$buildNumber, '
0054     'forceUpdate=$forceUpdate)';
0056   @override
0057   bool operator ==(Object other) =>
0058     identical(this, other) ||
0059     other is UpdateInfo &&
0060       version == other.version &&
0061       buildNumber == other.buildNumber &&
0062       forceUpdate == other.forceUpdate;
0064   @override
0065   int get hashCode => Object.hash(version, buildNumber, forceUpdate);
0066 }

// ===== 文件: lib/features/update/services/update_service.dart =====
0001 import 'dart:convert';
0003 import 'package:http/http.dart' as http;

```



```
0004 import 'package:package_info_plus/package_info_plus.dart';
0005 import 'package:yueyou/core/utils/cyber_logger.dart';
0007 import '../domain/update_info.dart';
0009 /// 热更新版本检测服务
0010 ///
0011 /// ## 工作流程
0012 /// 1. 通过 `PackageInfo.fromPlatform()` 获取本地版本信息
0013 /// 2. 向 `_versionApi` 发送 GET 请求, 获取服务端最新版本
0014 /// 3. 对比本地与服务端的 buildNumber (更精确, 不受多位语义版本影响)
0015 /// 4. 若服务端版本较新, 返回 [UpdateInfo]; 否则返回 null
0016 ///
0017 /// ## 版本对比策略
0018 /// 优先以 `buildNumber` 整数对比 (简单可靠);
0019 /// 若服务端响应无 `buildNumber`, 退级为语义版本字符串对比。
0020 ///
0021 /// ## API 接口约定
0022 /// - 请求: `GET https://<host>/version` (无鉴权, 公开端点)
0023 /// - 响应 200 + JSON:
0024 ///   ```json
0025 ///   {"version": "1.2.0", "buildNumber": 3, "forceUpdate": false,
0026 ///     "releaseNotes": "...", "downloadUrl": "https://..."}
0027 ///   ```
0028 /// - 超时: 5 秒 (网络异常时静默降级, 不中断用户)
0029 ///
0030 /// ## 注入方式
0031 /// API URL 通过 `--dart-define=UPDATE_API_URL=https://...` 注入,
0032 /// 默认为空字符串 (开发环境不发请求)。
0033 class UpdateService {
0034   UpdateService._();
0036   /// 编译时注入的版本检测 API URL
0037   static const String _versionApi = String.fromEnvironment(
0038     'UPDATE_API_URL',
0039     defaultValue: '',
0040   );
0042   /// 版本请求超时时间
0043   static const Duration _timeout = Duration(seconds: 5);
0045   // —— 公开接口 ——
0047   /// 检测是否有可用更新
0048   ///
0049   /// - 返回 [UpdateInfo]: 存在更新 (版本号大于本地)
0050   /// - 返回 `null`: 无更新、API 未配置或请求失败 (静默降级)
0051   static Future<UpdateInfo?> checkForUpdate() async {
0052     if (_versionApi.isEmpty) {
0053       return null;
0054     }
0056     try {
0057       final response = await http.get(Uri.parse(_versionApi)).timeout(_timeout);
0059       if (response.statusCode != 200) {
0060         CyberLogger.captureWarning(
0061           StateError('版本检测接口返回异常'),
```

```

0062         tag: 'update',
0063         extra: {
0064             'context': '版本检测 HTTP 响应',
0065             'statusCode': response.statusCode.toString(),
0066         },
0067     );
0068     return null;
0069 }
0071 final Map<String, dynamic> json =
0072     jsonDecode(response.body) as Map<String, dynamic>;
0073 final remote = UpdateInfo.fromJson(json);
0074 final local = await _getLocalVersion();
0075 if (_isNewerVersion(remote, local)) {
0076     CyberLogger.captureMessage(
0077         '发现新版本: remote=${remote.version}, local=${local.version}',
0078     );
0079     return remote;
0080 }
0081 CyberLogger.captureMessage('当前已是最新版本: ${local.version}');
0082 return null;
0083 } on Exception catch (e, stack) {
0084     // 网络异常、JSON 解析异常均静默降级, 不影响用户体验
0085     CyberLogger.captureWarning(
0086         e,
0087         stack: stack,
0088         tag: 'update',
0089         extra: {'context': '版本检测异常, 已静默降级'},
0090     );
0091     return null;
0092 }
0093 }
0094 }
0095 }
0097 // —— 内部工具 ——
0098 /// 获取本地安装版本信息 (仅读取, 无副作用)
0099 static Future<UpdateInfo> _getLocalVersion() async {
0100     final info = await PackageInfo.fromPlatform();
0101     final buildNum = int.tryParse(info.buildNumber) ?? 0;
0102     return UpdateInfo(
0103         version: info.version,
0104         buildNumber: buildNum,
0105         forceUpdate: false,
0106     );
0107 }
0108 }
0109 /// 判断 [remote] 版本是否比 [local] 更新
0110 ///
0111 /// 优先比较 buildNumber (整数, 精确);
0112 /// buildNumber 均为 0 时退级为语义版本字符串比较。
0113 static bool isNewerVersion(UpdateInfo remote, UpdateInfo local) =>
0114     _isNewerVersion(remote, local);
0115 static bool _isNewerVersion(UpdateInfo remote, UpdateInfo local) {
0116     // 以 buildNumber 优先对比

```

```

0119     if (remote.buildNumber > 0 || local.buildNumber > 0) {
0120         return remote.buildNumber > local.buildNumber;
0121     }
0122     // 降级：语义版本字符串对比（三段 int 比较）
0123     return _compareSemanticVersion(remote.version, local.version) > 0;
0124 }
0126 /// 语义版本比较，返回：正数 = a 更新，0 = 相同，负数 = b 更新
0127 static int compareSemanticVersion(String a, String b) =>
0128     _compareSemanticVersion(a, b);
0130 static int _compareSemanticVersion(String a, String b) {
0131     final aParts = _parseSemVer(a);
0132     final bParts = _parseSemVer(b);
0133     for (int i = 0; i < 3; i++) {
0134         final diff = aParts[i] - bParts[i];
0135         if (diff != 0) return diff;
0136     }
0137     return 0;
0138 }
0140 /// 将语义版本字符串拆分为三段整数列表（不足三段补 0）
0141 static List<int> _parseSemVer(String version) {
0142     final parts = version.split('.');
0143     return List.generate(3, (i) {
0144         if (i >= parts.length) return 0;
0145         return int.tryParse(parts[i]) ?? 0;
0146     });
0147 }
0148 }

```

```

// ===== 文件：lib/shared/widgets/cyber_confirm_dialog.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'package:yueyou/core/theme/cyber_colors.dart';
0003 import 'package:yueyou/core/theme/cyber_dimensions.dart';
0004 import 'package:yueyou/core/theme/cyber_text_styles.dart';
0005 import 'package:yueyou/shared/widgets/cyber_modal.dart';
0007 /// 赛博朋克风格确认对话框
0008 /// 用于重置游戏等需要二次确认的操作
0009 /// 基于 showCyberModal 封装，内部只提供确认内容（标题 + 消息 + 按钮）
0010 Future<bool> showCyberConfirmDialog({
0011     required BuildContext context,
0012     required String title,
0013     required String message,
0014     String confirmText = '确认',
0015     String cancelText = '取消',
0016 }) async {
0017     final result = await showCyberModal<bool>(<
0018         context: context,
0019         child: Padding(
0020             padding: const EdgeInsets.all(CyberDimensions.spacingL),
0021             child: Column(
0022                 mainAxisAlignment: MainAxisAlignment.min,

```

```

0023     children: [
0024         Text(
0025             title,
0026             style: CyberTextStyles.dialogTitle.copyWith(
0027                 color: CyberColors.whiteHigh,
0028                 fontWeight: FontWeight.w900,
0029             ),
0030         ),
0031         const SizedBox(height: CyberDimensions.spacingM),
0032         Flexible(
0033             child: SingleChildScrollView(
0034                 child: Text(
0035                     message,
0036                     textAlign: TextAlign.center,
0037                     style: CyberTextStyles.bodySmall.copyWith(
0038                         color: CyberColors.whiteMedium,
0039                         fontSize: 14,
0040                         height: 1.5,
0041                     ),
0042                 ),
0043             ),
0044         ),
0045         const SizedBox(height: CyberDimensions.spacingL),
0046         Row(
0047             children: [
0048                 Expanded(
0049                     child: _CyberButton(
0050                         text: cancelText,
0051                         isPrimary: false,
0052                         onTap: () => Navigator.of(context).pop(false),
0053                     ),
0054                 ),
0055                 const SizedBox(width: CyberDimensions.spacingMS),
0056                 Expanded(
0057                     child: _CyberButton(
0058                         text: confirmText,
0059                         isPrimary: true,
0060                         onTap: () => Navigator.of(context).pop(true),
0061                     ),
0062                 ),
0063             ],
0064         ),
0065     ],
0066 ),
0067 );
0068 );
0069 return result ?? false;
0070 }
0071
0072 class _CyberButton extends StatelessWidget {
0073     final String text;

```

```
0074   final bool isPrimary;
0075   final VoidCallback onTap;
0077   const _CyberButton({
0078     required this.text,
0079     required this.isPrimary,
0080     required this.onTap,
0081   });
0083   @override
0084   Widget build(BuildContext context) {
0085     return GestureDetector(
0086       onTap: onTap,
0087       child: Container(
0088         padding:
0089           const EdgeInsets.symmetric(vertical: CyberDimensions.spacingMS),
0090         decoration: BoxDecoration(
0091           gradient: isPrimary
0092             ? const LinearGradient(
0093               colors: [CyberColors.hotPink, CyberColors.neonPurple],
0094               begin: Alignment.topLeft,
0095               end: Alignment.bottomRight,
0096             )
0097             : null,
0098         color: isPrimary ? null : CyberColors.whiteSubtle,
0099         borderRadius: BorderRadius.circular(CyberDimensions.radiusS),
0100         border: Border.all(
0101           color:
0102             isPrimary ? CyberColors.transparent : CyberColors.whiteSubtle,
0103           width: CyberDimensions.borderNormal,
0104         ),
0105       ),
0106       child: Center(
0107         child: Text(
0108           text,
0109           style: CyberTextStyles.buttonLabel.copyWith(
0110             color: CyberColors.whiteHigh,
0111             fontWeight: FontWeight.w900,
0112           ),
0113         ),
0114       ),
0115     ),
0116   );
0117 }
0118 }
```

// ===== 文件: lib/shared/widgets/cyber_modal.dart =====

```
0001 import 'dart:ui';
0002 import 'package:flutter/material.dart';
0003 import 'package:yueyou/core/theme/cyber_colors.dart';
0004 import 'package:yueyou/core/theme/cyber_dimensions.dart';
0005 import 'package:yueyou/core/utils/cyber_performance_detector.dart';
```

```
0007 /// 赛博朋克风格全局弹窗封装
0008 /// 替代 Navigator.push 全屏跳转, 点击外部可关闭
0009 Future<T?> showCyberModal<T>({
0010   required BuildContext context,
0011   required Widget child,
0012   bool barrierDismissible = true,
0013 }) {
0014   return showGeneralDialog<T>(  
0015     context: context,  
0016     barrierDismissible: barrierDismissible,  
0017     barrierLabel: MaterialLocalizations.of(context).modalBarrierDismissLabel,  
0018     barrierColor: CyberColors.blackOverlay,  
0019     transitionDuration: CyberDimensions.animNormal,  
0020     pageBuilder: (context, animation, secondaryAnimation) {  
0021       return SafeArea(  
0022         child: Center(  
0023           child: Material(  
0024             type: MaterialType.transparency,  
0025             child: ScaleTransition(  
0026               scale: CurvedAnimation(  
0027                 parent: animation,  
0028                 curve: Curves.easeOutCubic,  
0029               ),  
0030             child: FadeTransition(  
0031               opacity: animation,  
0032               child: Container(  
0033                 margin: const EdgeInsets.symmetric(  
0034                   horizontal: CyberDimensions.spacingXL,  
0035                   vertical: 64,  
0036                 ),  
0037                 constraints: BoxConstraints(  
0038                   maxWidth: 500,  
0039                   maxHeight: MediaQuery.of(context).size.height * 0.85,  
0040                 ),  
0041                 decoration: BoxDecoration(  
0042                   borderRadius:  
0043                     BorderRadius.circular(CyberDimensions.radiusL),  
0044                   border: Border.all(  
0045                     color: CyberColors.neonCyan.withValues(alpha: 0.4),  
0046                     width: CyberDimensions.borderThick,  
0047                   ),  
0048                   boxShadow: [  
0049                     BoxShadow(  
0050                       color: CyberColors.neonCyan.withValues(alpha: 0.25),  
0051                       blurRadius: 30,  
0052                       spreadRadius: 2,  
0053                     ),  
0054                     const BoxShadow(  
0055                       color: CyberColors.blackShadow,  
0056                       blurRadius: 40,
```

```

0057         offset: Offset(0, 15),
0058     ),
0059 ],
0060 ),
0061 child: ClipRect(
0062     borderRadius: BorderRadius.circular(
0063         CyberDimensions.radiusL - CyberDimensions.borderThick,
0064     ),
0065     child: CyberPerformanceDetector.detectLevel() ==
0066         CyberAnimationLevel.low
0067       ? Container(
0068         color:
0069             CyberColors.glassDark.withValues(alpha: 0.98),
0070         child: child, // 直接渲染内容
0071       )
0072       : BackdropFilter(
0073         filter: ImageFilter.blur(
0074           sigmaX: CyberDimensions.blurStrong,
0075           sigmaY: CyberDimensions.blurStrong,
0076         ),
0077         child: Container(
0078           color: CyberColors.glassDark,
0079           child: child, // 移除了冗余的内部 ClipRect, 统一在外层精准截断
0080         ),
0081       ),
0082     ),
0083   ),
0084 ),
0085 ),
0086 ),
0087 ),
0088 );
0089 },
0090 );
0091 }

```

// ===== 文件: lib/shared/widgets/cyber_toast.dart =====

```

0001 import 'dart:async';
0002 import 'package:flutter/material.dart';
0003 import 'dart:ui';
0004 import '../core/theme/cyber_colors.dart';
0005 import '../core/theme/cyber_dimensions.dart';
0006 import '../core/theme/cyber_text_styles.dart';
0007 import '../core/utils/cyber_performance_detector.dart';
0008 import '../features/game_2048/presentation/widgets/board_mascot.dart';
0009 import '../main.dart';
0011 enum ToastType {
0012   info,
0013   error,
0014   success,

```

```
0015 }
0017 class CyberToast {
0018     static OverlayEntry? _currentEntry;
0019     static Timer? _currentTimer;
0021     static void show(
0022         String message, {
0023         ToastType type = ToastType.info,
0024         Duration duration = CyberDimensions.toastDuration,
0025     }) {
0026         // 移除之前的 Toast
0027         _removeCurrentEntry();
0029         // 创建新的 OverlayEntry
0030         final overlay = globalNavigatorKey.currentState?.overlay;
0031         if (overlay == null) return;
0032         final entry = OverlayEntry(
0033             builder: (context) => _CyberToastWidget(
0034                 message: message,
0035                 type: type,
0036                 onRemove: () => _currentEntry = null,
0037             ),
0038         );
0040         _currentEntry = entry;
0041         overlay.insert(entry);
0043         // 设置定时器, 自动移除
0044         _currentTimer?.cancel();
0045         _currentTimer = Timer(duration, () {
0046             _removeCurrentEntry();
0047         });
0048     }
0050     static void _removeCurrentEntry() {
0051         if (_currentEntry != null) {
0052             _currentEntry?.remove();
0053             _currentEntry = null;
0054         }
0055         _currentTimer?.cancel();
0056         _currentTimer = null;
0057     }
0058 }
0060 class _CyberToastWidget extends StatefulWidget {
0061     final String message;
0062     final ToastType type;
0063     final VoidCallback onRemove;
0065     const _CyberToastWidget({
0066         required this.message,
0067         required this.type,
0068         required this.onRemove,
0069     });
0071     @override
0072     __CyberToastWidgetState createState() => __CyberToastWidgetState();
0073 }
```



```
0075 class __CyberToastWidgetState extends State<__CyberToastWidget>
0076     with SingleTickerProviderStateMixin {
0077     late AnimationController _controller;
0078     late Animation<Offset> _slideAnimation;
0079     late Animation<double> _fadeAnimation;
0081     @override
0082     void initState() {
0083         super.initState();
0085         _controller = AnimationController(
0086             duration: CyberDimensions.animNormal,
0087             vsync: this,
0088         );
0090         _slideAnimation = Tween<Offset>(<
0091             begin: const Offset(0, -1),
0092             end: Offset.zero,
0093         ).animate(
0094             CurvedAnimation(
0095                 parent: _controller,
0096                 curve: Curves.easeOut,
0097             ),
0098         );
0100         _fadeAnimation = Tween<double>(<
0101             begin: 0.0,
0102             end: 1.0,
0103         ).animate(
0104             CurvedAnimation(
0105                 parent: _controller,
0106                 curve: Curves.easeOut,
0107             ),
0108         );
0110         _controller.forward();
0111     }
0113     @override
0114     void dispose() {
0115         _controller.dispose();
0116         super.dispose();
0117     }
0119     Color _getBorderColor() => switch (widget.type) {
0120         ToastType.error => CyberColors.neonPink,
0121         ToastType.success => CyberColors.neonGreen,
0122         ToastType.info => CyberColors.neonCyan
0123     };
0125     @override
0126     Widget build(BuildContext context) {
0127         return Positioned(
0128             top: MediaQuery.of(context).padding.top + CyberDimensions.spacingM,
0129             left: 0,
0130             right: 0,
0131             child: FadeTransition(
0132                 opacity: _fadeAnimation,
```

```

0133     child: SlideTransition(
0134       position: _slideAnimation,
0135       child: Align(
0136         alignment: Alignment.topCenter,
0137         child: IntrinsicWidth(
0138           child: Container(
0139             constraints: BoxConstraints(
0140               maxWidth: MediaQuery.of(context).size.width * 0.85,
0141             ),
0142             margin: const EdgeInsets.symmetric(
0143               horizontal: CyberDimensions.spacingML,
0144             ),
0145             decoration: BoxDecoration(
0146               borderRadius: BorderRadius.circular(CyberDimensions.radiusL),
0147               border: Border.all(
0148                 color: _getBorderColor(),
0149                 width: CyberDimensions.borderThick,
0150               ),
0151               boxShadow: [
0152                 BoxShadow(
0153                   color: _getBorderColor().withValues(alpha: 0.3),
0154                   blurRadius: 16,
0155                   offset: const Offset(0, 4),
0156                 ),
0157               ],
0158             ),
0159             child: ClipRRect(
0160               borderRadius: BorderRadius.circular(
0161                 CyberDimensions.radiusL - CyberDimensions.borderThick,
0162               ),
0163               child: CyberPerformanceDetector.detectLevel() ==
0164                 CyberAnimationLevel.low
0165                 ? Container(
0166                   color: CyberColors.glassDark.withValues(alpha: 0.98),
0167                   child: _buildContent(),
0168                 )
0169                 : BackdropFilter(
0170                   filter: ImageFilter.blur(sigmaX: 10, sigmaY: 10),
0171                   child: Container(
0172                     color: CyberColors.glassDark,
0173                     child: _buildContent(),
0174                   ),
0175                 ),
0176           ),
0177         ),
0178       ),
0179     ),
0180   ),
0181 ),
0182 );

```

```
0183 }
0185 Widget _buildContent() {
0186   return Padding(
0187     padding: const EdgeInsets.symmetric(
0188       horizontal: 16,
0189       vertical: 12,
0190     ),
0191     child: Row(
0192       mainAxisAlignment: MainAxisAlignment.min,
0193       crossAxisAlignment: CrossAxisAlignment.center,
0194       children: [
0195         // XIAOYO Avatar PFP
0196         Container(
0197           width: 44,
0198           height: 44,
0199           decoration: BoxDecoration(
0200             shape: BoxShape.circle,
0201             color: CyberColors.background.withValues(alpha: 0.8),
0202             border: Border.all(
0203               color: _getBorderColor().withValues(alpha: 0.7),
0204               width: 1,
0205             ),
0206             boxShadow: [
0207               BoxShadow(
0208                 color: _getBorderColor().withValues(alpha: 0.5),
0209                 blurRadius: 8,
0210               ),
0211             ],
0212           ),
0213         child: ClipOval(
0214           child: Transform.scale(
0215             scale: 0.65,
0216             // Transform.translate 去微调位置以确保脸部在圆圈正中心
0217             child: Transform.translate(
0218               offset: const Offset(0, -5),
0219               child: const IgnorePointer(
0220                 child: BoardMascot(),
0221               ),
0222             ),
0223           ),
0224         ),
0225       ],
0226       const SizedBox(width: 14),
0227       // Text Section
0228       Flexible(
0229         child: Column(
0230           mainAxisAlignment: MainAxisAlignment.min,
0231           crossAxisAlignment: CrossAxisAlignment.start,
0232           children: [
0233             Text(
```

```

0234         'XIAOYU',
0235         style: CyberTextStyles.captionBold.copyWith(
0236             color: _getBorderColor(),
0237             fontSize: 10,
0238             letterSpacing: 1.2,
0239         ),
0240     ),
0241     const SizedBox(height: 4),
0242     Text(
0243         widget.message,
0244         style: CyberTextStyles.bodySmallBold.copyWith(
0245             color: CyberColors.whiteHigh,
0246             height: 1.3,
0247         ),
0248         softWrap: true,
0249     ),
0250 ],
0251 ),
0252 ),
0253 ],
0254 ),
0255 );
0256 }
0257 }

```

// ===== 文件: lib/shared/widgets/safe_padding_wrap.dart =====

```

0001 import 'package:flutter/material.dart';
0002 import 'package:yueyou/core/theme/cyber_colors.dart';
0003 import 'package:yueyou/core/utils/safe_area_utils.dart';
0005 /// 顶部安全区包裹组件
0006 /// 严格执行“呼吸感”准则: SafeArea + 额外 24.0px 的 Padding
0007 class SafePaddingWrap extends StatelessWidget {
0008     final Widget child;
0009     const SafePaddingWrap({
0010         super.key,
0011         required this.child,
0012     });
0013     @override
0014     Widget build(BuildContext context) {
0015         return Container(
0016             color: CyberColors.background,
0017             child: SafeArea(
0018                 bottom: false, // 顶部组件不强制底部安全区
0019                 child: Padding(
0020                     padding: const EdgeInsets.only(
0021                         top: SafeAreaUtils.topBreathPadding,
0022                         left: 16.0,
0023                         right: 16.0,
0024                     ),
0025                     child: child,

```

```

0028     ),
0029     ),
0030   );
0031 }
0032 }

// ===== 文件: lib/shared/widgets/tts_error_listener.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'package:yueyou/core/utils/cyber_logger.dart';
0003 import 'package:flutter_riverpod/flutter_riverpod.dart';
0004 import '../features/audio/domain/tts_audio_state.dart';
0005 import '../features/audio/providers/tts_audio_notifier.dart';
0006 import 'cyber_toast.dart';
0008 /// TTS 错误全局监听器组件
0009 ///
0010 /// 将任意页面或整个 App 根节点包裹在此组件下,
0011 /// 每当 [TtsAudioError] 产生新时间戳时,
0012 /// 自动弹出 [CyberToast] 错误提示, 无需各页面手动监听。
0013 class TtsErrorListener extends ConsumerStatefulWidget {
0014   final Widget child;
0015   const TtsErrorListener({super.key, required this.child});
0017   @override
0018   ConsumerState<TtsErrorListener> createState() => _TtsErrorListenerState();
0019 }
0021 class _TtsErrorListenerState extends ConsumerState<TtsErrorListener> {
0022   int _previousErrorTime = 0;
0023   String? _previousFallback;
0024   int _lastFallbackTimestamp = 0;
0026   @override
0027   void didChangeDependencies() {
0028     super.didChangeDependencies();
0029     // 依赖变化时刷新快照, 避免初始化重复触发旧错误
0030     final audioState = ref.read(ttsAudioProvider);
0031     _previousErrorTime = switch (audioState) {
0032       TtsAudioError(:final timestamp) => timestamp,
0033       TtsAudioIdle() ||
0034       TtsAudioBuffering() ||
0035       TtsAudioPlaying() ||
0036       TtsAudioPaused() =>
0037         0,
0038     };
0039     _previousFallback = audioState.fallbackMessage;
0040   }
0042   @override
0043   Widget build(BuildContext context) {
0044     final audioState = ref.watch(ttsAudioProvider);
0045     final (:err, :timestamp) = switch (audioState) {
0046       TtsAudioError(:final message, :final timestamp) => (
0047         err: message,
0048         timestamp: timestamp,

```

```
0049     ),
0050     TtsAudioIdle() ||
0051     TtsAudioBuffering() ||
0052     TtsAudioPlaying() ||
0053     TtsAudioPaused() =>
0054     (err: null, timestamp: 0),
0055 };
0056 final currentTimestamp = DateTime.now().millisecondsSinceEpoch;
0057 if (err != null && timestamp != _previousErrorTime) {
0058     _previousErrorTime = timestamp;
0059     CyberLogger.captureMessage('[TtsErrorListener] 检测到新错误, 将展示 CyberToast');
0060     WidgetsBinding.instance.addPostFrameCallback((_) {
0061         if (!mounted) return;
0062         try {
0063             CyberToast.show(err, type: ToastType.error);
0064         } catch (e, stack) {
0065             CyberLogger.captureWarning(
0066                 e,
0067                 stack: stack,
0068                 tag: 'tts',
0069                 extra: {'context': 'TtsErrorListener 展示错误 Toast 失败'},
0070             );
0071         }
0072     });
0073 }
0074 }
0075 final fallback = audioState.fallbackMessage;
0076 if (fallback == null) {
0077     _previousFallback = null;
0078 } else {
0079     final isSameFallback = fallback == _previousFallback;
0080     final isThrottled =
0081         isSameFallback && (currentTimestamp - _lastFallbackTimestamp < 1000);
0082     if (!isThrottled) {
0083         _previousFallback = fallback;
0084         _lastFallbackTimestamp = currentTimestamp;
0085         WidgetsBinding.instance.addPostFrameCallback((_) {
0086             if (!mounted) return;
0087             try {
0088                 CyberToast.show(fallback, type: ToastType.info);
0089             } catch (e, stack) {
0090                 CyberLogger.captureWarning(
0091                     e,
0092                     stack: stack,
0093                     tag: 'tts',
0094                     extra: {'context': 'TtsErrorListener 展示降级通知 Toast 失败'},
0095                 );
0096             }
0097         });
0098     }
0099 }
0100 }
0101 }
```

```
0103     return widget.child;
0104 }
0105 }
```

```
// ===== 文件: server/main.go =====
```

```
0001 package main
0003 import (
0004     "log"
0006     "github.com/gin-gonic/gin"
0007 )
0009 func main() {
0010     gin.SetMode(gin.ReleaseMode)
0011     r := gin.Default()
0012     if err := r.SetTrustedProxies(nil); err != nil {
0013         log.Fatal(err)
0014     }
0016     // —— TTS 语音合成 ——
0017     r.POST("/api/v1/tts", ttsHandler)
0019     // —— 书籍服务 ——
0020     r.GET("/api/v1/book/catalog", bookCatalogHandler)
0021     r.POST("/api/v1/book/chapter", bookChapterHandler)
0023     // —— 合规页面 ——
0024     r.GET("/privacy", privacyHandler)
0026     log.Println("yueyou-server on :8081")
0027     r.Run(":8081")
0028 }
```

```
// ===== 文件: server/config.go =====
```

```
0001 package main
0003 import "os"
0005 // Config 持有所有运行时配置，优先从环境变量读取，回退到内置默认值。
0006 // 生产部署：通过 systemd EnvironmentFile 或 Docker --env-file 注入。
0007 type Config struct {
0008     AKID      string // 阿里云 AccessKey ID
0009     AKSecret   string // 阿里云 AccessKey Secret
0010     OSSEndpoint string // OSS 内网端点（服务器与 OSS 同 region 时用内网省流量）
0011     OSSBucket  string // OSS Bucket 名称
0012     OSSPub     string // OSS / CDN 公网访问前缀（无尾斜线）
0013 }
0015 // cfg 为全局单例配置，在 main() 启动前由 init() 初始化。
0016 var cfg Config
0018 func init() {
0019     cfg = Config{
0020         AKID:      getenv("YUEYOU_AKID", ""),
0021         AKSecret:   getenv("YUEYOU_AKSEC", ""),
0022         OSSEndpoint: getenv("YUEYOU_OSS_EP", "oss-cn-beijing-internal.aliyuncs.com"),
0023         OSSBucket:  getenv("YUEYOU_OSS_BKT", "general-storage"),
0024         OSSPub:     getenv("YUEYOU_OSS_PUB", "https://general-storage.oss-cn-beijing.aliyuncs.com")
0025     }
0026 }
```

```

0028 func getenv(key, fallback string) string {
0029     if v := os.Getenv(key); v != "" {
0030         return v
0031     }
0032     return fallback
0033 }

// ===== 文件: server/handler_book.go =====
0001 package main
0003 import (
0004     _ "embed"
0005     "fmt"
0006     "log"
0008     "github.com/gin-gonic/gin"
0009 )
0011 // xiyoujiCatalogJSON 在编译期嵌入目录文件, 换版本时只需替换 data/xiyouji_catalog.json 并重新编译。
0012 //
0013 //go:embed data/xiyouji_catalog.json
0014 var xiyoujiCatalogJSON []byte
0016 // xiyoujiTotalChapters 总章节数, 与客户端 BookConstants.defaultTotalChapters 严格对齐。
0017 const xiyoujiTotalChapters = 100
0019 // bookCatalogHandler 返回西游记目录(章节标题列表)。
0020 //
0021 // GET /api/v1/book/catalog?bookId=xiyouji
0022 //
0023 // 响应格式: {"status": "success", "chapters": [{"title": "...", "lineIndex": 0}...]}
0024 func bookCatalogHandler(c *gin.Context) {
0025     bookID := c.Query("bookId")
0026     if bookID != "xiyouji" {
0027         c.JSON(404, gin.H{"status": "error", "message": "书籍不存在"})
0028         return
0029     }
0030     // 直接返回嵌入的 JSON, 客户端按约定解析
0031     c.Data(200, "application/json; charset=utf-8", xiyoujiCatalogJSON)
0032 }
0034 // bookChapterHandler 派发章节 CDN 下载地址, 遵循分离下载原则。
0035 //
0036 // POST /api/v1/book/chapter
0037 // 请求体: {"bookId": "xiyouji", "chapterIndex": 0}
0038 // 响应体: {"status": "success", "url": "https://cdn.../books/chapters/xiyouji/001.txt"}
0039 //
0040 // 客户端收到 URL 后自行 GET 下载章节正文, 与 TTS 分离下载完全对齐。
0041 func bookChapterHandler(c *gin.Context) {
0042     var req struct {
0043         BookID      string `json:"bookId" binding:"required"`
0044         ChapterIndex int    `json:"chapterIndex"`
0045     }
0046     if err := c.ShouldBindJSON(&req); err != nil {
0047         c.JSON(400, gin.H{"status": "error", "message": "参数错误"})
0048         return

```



```

0049     }
0050     if req.BookID != "xiyouji" {
0051         c.JSON(404, gin.H{"status": "error", "message": "书籍不存在"})
0052         return
0053     }
0054     if req.ChapterIndex < 0 || req.ChapterIndex >= xiyoujiTotalChapters {
0055         c.JSON(400, gin.H{"status": "error", "message": "章节索引越界"})
0056         return
0057     }
0059     // chapterIndex 0-based, OSS 文件名 1-based 三位补零
0060     filename := fmt.Sprintf("%03d.txt", req.ChapterIndex+1)
0061     url := fmt.Sprintf("%s/books/xiyouji/%s", cfg.OSSPub, filename)
0063     log.Printf("[Book] 派发章节: index=%d -> %s", req.ChapterIndex, url)
0065     c.JSON(200, gin.H{
0066         "status": "success",
0067         "url":    url,
0068     })
0069 }

// ===== 文件: server/handler_privacy.go =====
0001 package main
0003 import (
0004     "net/http"
0006     "github.com/gin-gonic/gin"
0007 )
0009 const privacyPolicyHTML = `<!doctype html>
0010 <html lang="zh-CN">
0011 <head>
0012     <meta charset="utf-8">
0013     <meta name="viewport" content="width=device-width, initial-scale=1">
0014     <title>阅游隐私政策</title>
0015     <style>
0016         body { margin: 0; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
0017         main { max-width: 880px; margin: 0 auto; padding: 32px 18px 64px; }
0018         h1, h2 { color: #22d3ee; }
0019         h1 { border-bottom: 1px solid rgba(34, 211, 238, .35); padding-bottom: 12px; }
0020         section { margin-top: 28px; padding: 18px; border: 1px solid rgba(34, 211, 238, .18); border-
0021         a { color: #67e8f9; }
0022         .muted { color: rgba(229, 250, 255, .72); }
0023     </style>
0024 </head>
0025 <body>
0026 <main>
0027     <h1>阅游隐私政策</h1>
0028     <p class="muted">更新日期: 2026年5月4日 | 生效日期: 2026年5月4日</p>
0030     <section>
0031         <h2>一、开发者信息</h2>
0032         <p>开发者/运营主体: 胡传龙</p>
0033         <p>联系邮箱: <a href="mailto:hucloong@163.com">hucloong@163.com</a></p>
0034     </section>

```

```

0036 <section>
0037     <h2>二、我们如何收集和使用信息</h2>
0038     <p>阅游是一款本地小说阅读、听读与 2048 益智游戏应用。我们不会收集通讯录、短信、通话记录、定位、相机、麦克风等敏感个人信息。</p>
0039     <p>当您使用云端 TTS 朗读功能时，应用会将当前需要朗读的文本片段发送至 TTS 服务，用于生成音频。除当前朗读文本外，我们不会上传您的书籍。</p>
0040 </section>
0042 <section>
0043     <h2>三、本地数据存储</h2>
0044     <p>阅读进度、书架、设置、游戏存档和 TTS 缓存文件仅保存在您的设备本地。您可以通过卸载应用或清除应用数据删除这些本地数据。</p>
0045 </section>
0047 <section>
0048     <h2>四、文件访问说明</h2>
0049     <p>导入 TXT 小说时，阅游通过 Android 系统文件选择器访问您主动选择的文件。应用不会扫描您的其他目录，也不会未经您选择的情况下读取其他文件。</p>
0050 </section>
0052 <section>
0053     <h2>五、第三方服务</h2>
0054     <p>阅游可能使用云端 TTS 服务、对象存储/CDN 服务提供音频合成、默认书籍目录与章节下载能力。这些服务仅用于实现应用功能，不用于个人画像。</p>
0055 </section>
0057 <section>
0058     <h2>六、权限使用说明</h2>
0059     <p>网络权限用于访问 TTS 与书籍服务。文件导入通过系统文件选择器完成授权，不申请读取全部存储空间权限。</p>
0060 </section>
0062 <section>
0063     <h2>七、未成年人保护</h2>
0064     <p>阅游不面向低龄儿童提供定向服务。如未成年人使用本应用，应在监护人指导下使用。</p>
0065 </section>
0067 <section>
0068     <h2>八、您的权利</h2>
0069     <p>如您需要咨询隐私问题、反馈问题或请求删除相关数据，可通过邮箱 <a href="mailto:hucloong@163.com">hucloong@163.com</a> 与我们联系。</p>
0070 </section>
0072 <section>
0073     <h2>九、政策更新</h2>
0074     <p>我们可能根据功能变化更新本隐私政策。更新后将通过页面日期或应用内提示展示。</p>
0075 </section>
0076 </main>
0077 </body>
0078 </html>`
0080 // privacyHandler 返回阅游隐私政策静态页面。
0081 func privacyHandler(c *gin.Context) {
0082     c.Data(http.StatusOK, "text/html; charset=utf-8", []byte(privacyPolicyHTML))
0083 }

// ===== 文件: server/handler_tts.go =====
0001 package main
0002 import (
0003     "bytes"
0004     "crypto/md5"
0005     "fmt"
0006     "log"
0007     "os"
0008     "os/exec"

```

```

0010     "sync"
0012     "github.com/aliyun/aliyun-oss-go-sdk/oss"
0013     "github.com/gin-gonic/gin"
0014 )
0016 // edgeTtsSem 严格串行化 edge-tts 进程，避免触发微软服务端并发任务限制。
0017 var edgeTtsSem = make(chan struct{}, 1)
0019 // inflightMu 保护 inflightKeys，用于对同一文本的并发请求去重。
0020 var (
0021     inflightMu sync.Mutex
0022     inflightKeys = map[string]chan struct{} {}
0023 )
0025 // ttsHandler 接受文本和音色，生成 MP3 并上传 OSS，返回 CDN 播放地址。
0026 // 遵循分离下载原则：客户端 POST 获取 URL，再自行 GET 下载音频。
0027 func ttsHandler(c *gin.Context) {
0028     var req struct {
0029         Text string `json:"text" binding:"required"`
0030         Voice string `json:"voice"`
0031     }
0032     if err := c.ShouldBindJSON(&req); err != nil {
0033         c.JSON(400, gin.H{"status": "error", "message": "JSON error"})
0034         return
0035     }
0036     if req.Voice == "" {
0037         req.Voice = "zh-CN-XiaoxiaoNeural"
0038     }
0040     hash := md5.Sum([]byte(req.Text + req.Voice))
0041     fn := fmt.Sprintf("cache/%x.mp3", hash)
0042     fu := fmt.Sprintf("%s/%s", cfg.OSSPub, fn)
0044     bk, err := ossBucket()
0045     if err != nil {
0046         c.JSON(500, gin.H{"status": "error", "message": "OSS 初始化失败"})
0047         return
0048     }
0050     // 缓存命中直接返回
0051     if ok, _ := bk.IsObjectExist(fn); ok {
0052         c.JSON(200, gin.H{"status": "success", "url": fu})
0053         return
0054     }
0056     // 同一文本正在合成时，等待已有任务完成后复用 OSS 缓存，避免重复调用 edge-tts
0057     inflightMu.Lock()
0058     if ch, ok := inflightKeys[fn]; ok {
0059         inflightMu.Unlock()
0060         <-ch
0061         if ok2, _ := bk.IsObjectExist(fn); ok2 {
0062             c.JSON(200, gin.H{"status": "success", "url": fu})
0063         } else {
0064             c.JSON(500, gin.H{"status": "error", "message": "语音合成失败"})
0065         }
0066         return
0067     }

```

```

0068     doneCh := make(chan struct{})
0069     inflightKeys[fn] = doneCh
0070     inflightMu.Unlock()
0071     defer func() {
0072         close(doneCh)
0073         inflightMu.Lock()
0074         delete(inflightKeys, fn)
0075         inflightMu.Unlock()
0076     }()
0077     log.Printf("[TTS] 合成: %s", req.Text)
0078     // 获取信号量, 严格串行化 edge-tts 进程
0079     edgeTtsSem <- struct{}{}
0080     defer func() { <-edgeTtsSem }()
0081     tmpFile := fmt.Sprintf("/tmp/tts_%x.mp3",
0082         md5.Sum([]byte(req.Text+req.Voice+fmt.Sprintf("%d", os.Getpid()))))
0083     cmd := exec.Command("edge-tts",
0084         "--text", req.Text,
0085         "--voice", req.Voice,
0086         "--write-media", tmpFile,
0087     )
0088     if out, err := cmd.CombinedOutput(); err != nil {
0089         log.Printf("[TTS] edge-tts 错误: %v, output: %s", err, string(out))
0090         c.JSON(500, gin.H{"status": "error", "message": "语音合成失败"})
0091         return
0092     }
0093     audioData, err := os.ReadFile(tmpFile)
0094     os.Remove(tmpFile)
0095     if err != nil {
0096         c.JSON(500, gin.H{"status": "error", "message": "读取音频失败"})
0097         return
0098     }
0099     if len(audioData) < 100 {
0100         c.JSON(500, gin.H{"status": "error", "message": "生成音频为空"})
0101         return
0102     }
0103     if err := bk.PutObject(fn, bytes.NewReader(audioData)); err != nil {
0104         c.JSON(500, gin.H{"status": "error", "message": "OSS 上传失败"})
0105         return
0106     }
0107     c.JSON(200, gin.H{"status": "success", "url": fu})
0108 }
0109 // ossBucket 返回配置好的 OSS bucket 客户端。
0110 func ossBucket() (*oss.Bucket, error) {
0111     cli, err := oss.New(cfg.OSSEndpoint, cfg.AKID, cfg.AKSecret)
0112     if err != nil {
0113         return nil, err
0114     }
0115     return cli.Bucket(cfg.OSSBucket)
0116 }

```